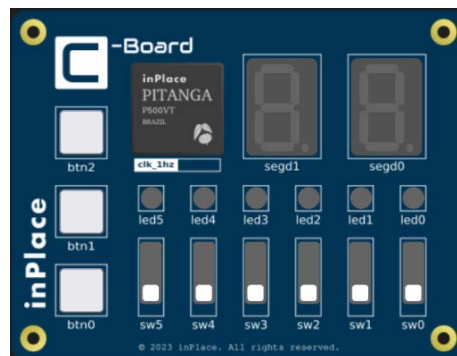


Guia do Iniciante Pitanga

Versão V0.1

Versão Rascunho Preliminar



Sumário

1. Sobre este manual	4
2. O que é a Pitanga?	4
2.1. Visão Geral	4
2.2. Área do Verilog	4
2.3. Área do pinout	6
2.4. Área do Relatório	6
2.5. Área da placa.....	8
3. O que precisa para fazer um circuito?	9
3.1. Sobre o Verilog.....	9
3.2. Sobre o Pinout.....	9
4. Exemplos: Como fazer um circuito?.....	9
4.1. Primeiro Exemplo: uma entrada e uma saída.....	9
4.2. Segundo exemplo: uma entrada e duas saídas	11
4.3. Terceiro exemplo: duas entradas e duas saídas	12
4.4. Quarto exemplo: três entradas e duas saídas	13
4.5. Quinto exemplo: somador hierárquico de dois bits	15
5. Entradas e saídas na placa	18
5.1. Botões e como usá-los	18
5.2. Chaves e como usá-las	19
5.3. Relógios e como usá-los.....	21
5.4. Entradas constantes e como usá-las.....	22
5.5. Leds e como usá-los	23
5.6. Displays e como usá-los	24
6. Circuitos sequenciais: Flip-flops	30
7. Os planos de assinatura e os tipos de placa	36
8. Uso da plataforma Pitanga.....	38
8.1. Como se logar?.....	38
8.2. Como obter ajuda?	39
8.3. Iniciando e parando a emulação	40
8.4. Outros comandos da Pitanga.....	41
9. IA Assistiva Neuralis	43

10. Exemplo do Sistema explicado linha a linha	45
11. Boas práticas e estilos de Verilog sem suporte na placa	48
11.1. O Verilog não suporta laços combinacionais	48
11.2. Relógios de Flip-flops não suportam lógica	48
11.3. A placa não suporta múltiplos relógios automáticos	49
11.4. Pinos de set e reset não suportam lógica	50
11.5. Set e reset não podem ser ambos assíncronos	50
11.6. Usar <i>always@(*)</i> não é boa prática	50
12. Plataforma Pitanga como substituição de <i>protoboards</i>	51
13. Referências comentadas	51
14. Links externos	52
15. Histórico de versões	52
16. Declaração de copyright	52

1. Sobre este manual

Este manual descreve o Sistema Pitanga, restringindo-se à placa virtual. Embora exista a possibilidade de uso de placa física, este manual se restringe a versão virtual. Aqui o foco é mais o uso do sistema Pitanga, assim espera-se que o leitor já saiba (ou busque em outras fontes) material introdutório sobre circuitos digitais e sua descrição em linguagem Verilog.

2. O que é a Pitanga?

A plataforma Pitanga é um sistema de emulação de circuitos digitais descritos em linguagem Verilog. Na prática o sistema permite ao usuário ter uma experiência bastante similar ao uso de uma placa de prototipação física, mas usando uma placa virtual com acesso remoto.

2.1. Visão Geral

A Figura 1 apresenta as quatro principais áreas da plataforma Pitanga, a saber: Verilog, Pinout, relatório e placa. Estas áreas são discutidas nas próximas seções.



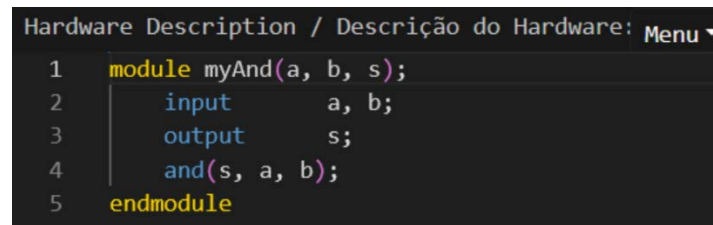
Figura 1: As quatro principais áreas da plataforma Pitanga.

2.2. Área do Verilog

A área do Verilog é uma janela de texto, onde um circuito digital pode ser descrito em linguagem Verilog. Deste modo, esta área é onde será digitado, copiado, editado ou lido o texto da descrição do circuito digital em Verilog.

A Figura 2 apresenta um exemplo da descrição Verilog de uma porta and de duas entradas. Apesar de este manual não ser uma descrição da sintaxe Verilog, vamos fazer uma breve descrição do exemplo. A seguir, cada linha do exemplo na Figura 2 é discutida e explicada.

A linha 1 descreve um módulo (palavra chave **module**, que significa módulo em inglês) chamado **myAnd**. Este módulo tem três pinos chamados **a**, **b** e **s**, descritos entre parênteses, como parâmetros do módulo. A linha 1 termina em ponto e vírgula, pois isto é parte da sintaxe Verilog.



```
Hardware Description / Descrição do Hardware: Menu ▾
1  module myAnd(a, b, s);
2      input    a, b;
3      output   s;
4      and(s, a, b);
5  endmodule
```

Figura 2: A área do Verilog na plataforma Pitanga.

A linha 2 usa a palavra chave **input** (que significa entrada em inglês) para dizer que os pinos **a** e **b** são pinos de entrada. Note que a declaração destes pinos como parâmetros no corpo do módulo na linha 1 não dizia se os pinos eram de entrada ou de saída.

De modo bastante similar, a linha 3 usa a palavra chave **output** (que significa saída em inglês) para dizer que o pino **s** é um pino de saída. Note que a declaração de **s** como parâmetro no corpo do módulo na linha 1 não dizia se o pino era de entrada ou de saída.

A linha 4 define a lógica do circuito. A palavra chave **and** é usada para criar uma porta lógica and de duas entradas. Note que a chamada da and é feita com três parâmetros porque uma porta and de duas entradas tem duas entradas e uma saída, totalizando três parâmetros. A sintaxe Verilog sempre assume que o primeiro parâmetro da chamada de uma porta lógica primitiva é a saída da célula, assim a sintaxe Verilog é entendida como **and(nomeSaída, nomeEntrada1, nomeEntrada2)**.

A linha 5 indica o fim do módulo que havia sido iniciado na linha 1. A palavra chave usada para terminar o módulo é **endmodule** em Verilog. A palavra **endmodule** pode ser traduzida como *fimdomódulo* em português.

A Figura 3 apresenta as opções do **Menu** no canto superior direito da área do Verilog. Através deste menu é possível abrir um arquivo Verilog pré-existente (opção **Open**), salvar um arquivo Verilog (opção **Save**), ou recarregar o exemplo padrão da placa Pitanga (opção **Reload Example**).

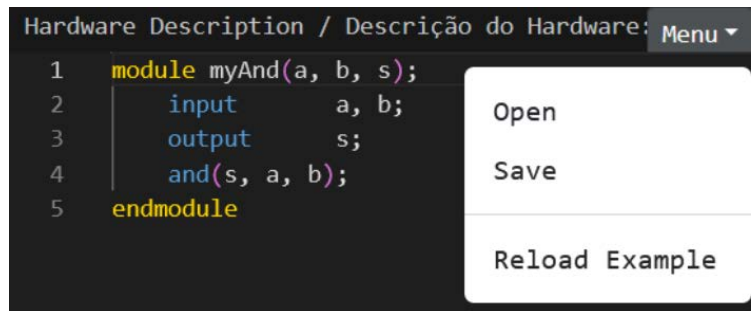


Figura 3: A área do Verilog na plataforma Pitanga.

2.3. Área do pinout

A área do pinout também é uma janela de texto, onde é feita a descrição de como as entradas e saídas do módulo Verilog se conectam aos pinos da placa virtual Pitanga. Esta informação não está contida no Verilog, então é necessário fornecer um segundo arquivo do tipo **pinout** para fazer esta associação.

Um exemplo de arquivo de pinout para a porta and de duas entradas descrita anteriormente é apresentado na figura 4. A linha 1 diz que o pino de entrada **a** do Verilog está associado com a chave **sw1** da placa. A linha 2 diz que o pino de entrada **b** do Verilog está associado com a chave **sw0** da placa. A linha 3 diz que o pino de saída **s** do Verilog está associado com o led **led0** da placa. Os nomes **sw1**, **sw0** e **led0** são indicados na serigrafia da placa virtual Pitanga e serão descritos em maior detalhe mais adiante neste manual.

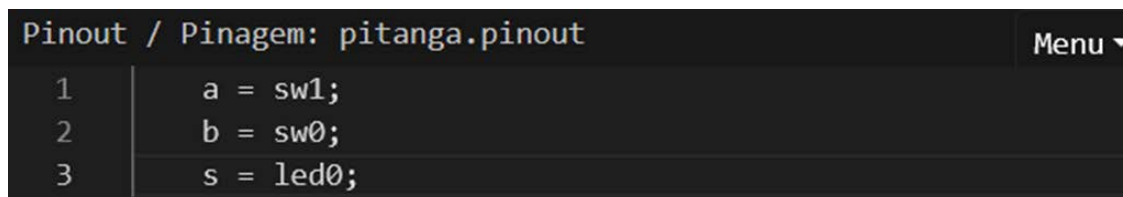


Figura 4: A área do pinout na plataforma Pitanga.

O **Menu** no canto superior direito da Figura 4 apresenta opções similares ao **Menu** no canto superior direito da área do Verilog (mostradas na Figura 3). Através deste menu é possível abrir um arquivo **.pinout** pré-existente (opção **Open**), salvar um arquivo **.pinout** (opção **Save**), ou recarregar o arquivo **.pinout** exemplo padrão da placa Pitanga (opção **Reload Example**).

2.4. Área do Relatório

A área do relatório apresenta dados do circuito após a compilação do conjunto dos arquivos Verilog e seu arquivo **.pinout** associado com sucesso. No caso da porta lógica **and** de duas entradas discutida nas seções anteriores, o relatório de compilação é apresentado na Figura 5. Como esperado, apenas uma porta lógica **and** de duas entradas é usada (**AND2 1**, no canto superior esquerdo da Figura 5), já que o

circuito descrito é uma porta *and* de duas entradas. A contagem das outras portas possíveis depois que o circuito foi mapeado aparece zerada (por exemplo, **XOR2 0**, no canto superior direito da Figura 5). A lista das portas lógicas disponíveis em um circuito mapeado na biblioteca de células da plataforma Pitanga é apresentada na Tabela 1.

AND2	1	NAND2	0	XOR2	0
AND3	0	NAND3	0	XOR3	0
AND4	0	NAND4	0	XOR4	0
OR2	0	NOR2	0	XNOR2	0
OR3	0	NOR3	0	XNOR3	0
OR4	0	NOR4	0	XNOR4	0
BUF	0	INV	0	DFFRSE	0
Cells utilization: 1					
Transistor count : 6/200 transistors (3.00 %)					

Figura 5: A área do relatório na plataforma Pitanga.

Tabela 1: Portas lógicas disponíveis em um circuito mapeado na biblioteca de células da plataforma Pitanga.

Nome da porta	Descrição
AND2, AND3 e AND4	Portas lógicas AND de 2, 3 e 4 entradas.
OR2, OR3 e OR4	Portas lógicas OR de 2, 3 e 4 entradas.
NAND2, NAND3 e NAND4	Portas lógicas NAND de 2, 3 e 4 entradas.
NOR2, NOR3 e NOR4	Portas lógicas NOR de 2, 3 e 4 entradas.
XOR2, XOR3 e XOR4	Portas lógicas XOR de 2, 3 e 4 entradas.
XNOR2, XNOR3 e XNOR4	Portas lógicas XNOR de 2, 3 e 4 entradas.
BUF	Porta lógica buffer (copia a entrada para a saída)
INV	Porta lógica inversora (copia a entrada para a saída)
DFFRSE	Flip-flop tipo D com Reset, Set e Enable

As duas linhas finais do relatório apresentado na Figura 5 descrevem a quantidade de células utilizadas, bem como a contagem do número de transistores em relação ao total de transistores disponível na placa virtual Pitanga. Esta descrição é tecnicamente densa e não cabe discuti-la em detalhe neste manual. Ainda assim, vamos explicar brevemente as duas últimas linhas.

A linha **Cells utilization: 1** indica que apenas uma porta lógica foi utilizada. Aqui se usa a palavra **Cell** porque em um contexto de síntese física e fabricação do circuito integrado a porta lógica será uma *célula* (*Cell*, em inglês) de uma biblioteca de células usada no leiaute físico do circuito integrado.

A linha **Transistor count: 6/200 transistors (3.00%)** indica que foram utilizados seis transistores dos 200 transistores disponíveis na placa Pitanga **T-board** (versão trial). Isto acontece porque em uma tecnologia CMOS, que é a tecnologia mais usada em circuitos digitais integrados, uma porta lógica and de duas entradas tem seis transistores como blocos construtivos internos.

2.5. Área da placa

A área da placa apresenta um circuito funcional sendo emulado na placa **T-board** (versão trial), após a compilação do conjunto dos arquivos Verilog e seu arquivo **.pinout** associado com sucesso. No caso da porta lógica **and** de duas entradas, discutida nas seções anteriores, a área da placa virtual é apresentada na Figura 6.

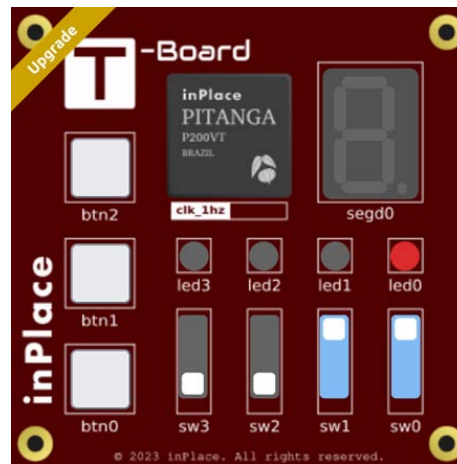


Figura 6: A área da placa virtual na plataforma Pitanga.

É possível verificar na Figura 6 que a placa **T-board** tem quatro leds, com nomes **led3**, **led2**, **led1** e **led0**, conforme a serigrafia da placa. Destes, apenas o **led0** está aceso, como representa o fato de ele apresentar uma cor vermelha distinta dos outros leds. Vamos entender porque isto acontece. Considerando-se o arquivo **.pinout** apresentado na Figura 4, sabemos que a entrada **a** é controlada pela chave **sw1** e a entrada **b** é controlada pela chave **sw0**. O fato de as chaves que controlam **a** e **b** estarem para cima faz com que **a=1** e **b=1**. Deste modo $s=a \cdot b=1 \cdot 1=1$, ou seja: **s=1**. Como a saída **s** é mostrada no **led0**, o **led0** acende. O usuário da placa virtual pode apagar o **led0** movendo uma das chaves **sw1** ou **sw0** para baixo e alterando o valor de **a** ou de **b**, de modo a fazer **s=0**, o que apagaria o **led0**. Assim, o usuário interage com a placa virtual de modo similar a interação com uma placa física.

Observando a Figura 6, é possível perceber que a placa **T-board** apresenta como dispositivos de entrada três botões (**btn2**, **btn1** e **btn0**), quatro chaves (**sw3**, **sw2**, **sw1** e **sw0**) e um sinal de relógio de 1Hz (**clk_1hz**). Os dispositivos de saída da placa **T-board**, conforme a Figura 6, consistem em 4 leds (**led3**, **led2**, **led1** e **led0**) e um display de sete segmentos (**segd0**) que inclui um ponto decimal. O uso destes diferentes dispositivos de entrada e saída é feito através de especificação de pinos no o arquivo **.pinout**. A próxima seção começa a detalhar melhor o uso dos pinos da placa através de alguns exemplos.

3. O que precisa para fazer um circuito?

Para fazer um circuito na placa virtual Pitanga duas definições são necessárias: a funcionalidade do circuito e a conexão dos pinos. A definição da funcionalidade do circuito é feita através de um arquivo **Verilog** que descreve o hardware. A definição da conexão dos pinos é feita através do arquivo **.pinout**. A seguir estes dois tipos de arquivo são descritos em maior detalhe, com alguns exemplos.

3.1. Sobre o Verilog

O arquivo Verilog descreve um hardware, já que Verilog é uma linguagem de descrição de hardware. É importante perceber que a interpretação da linha **and(s, a, b);** deve ser entendida como se fosse: “Existe uma porta lógica do tipo and cuja saída é **s** e cujas entradas são **a** e **b**”. Deste modo, a ordem com que as portas lógicas são descritas dentro de um módulo não afeta o funcionamento do circuito.

3.2. Sobre o Pinout

O assinalamento dos pinos feito através do arquivo **.pinout** afeta como o usuário interage com o circuito na placa virtual. Um erro bastante comum é não colocar os bits mais significativos e menos significativos na ordem correta tanto nos dispositivos de entrada (chaves) quanto nos dispositivos de saída (leds). Estes erros de interpretação podem levar o usuário a tentar encontrar no circuito (arquivo Verilog) um erro que na verdade se encontra no assinalamento dos pinos (arquivo **.pinout**).

4. Exemplos: Como fazer um circuito?

A seguir descrevemos alguns pares de Verilog e **.pinout** que podem ser usados para entender melhor a plataforma Pitanga. Para o primeiro exemplo são apresentadas capturas de tela que indicam como fazer a compilação.

4.1. Primeiro Exemplo: uma entrada e uma saída

O exemplo mais simples de um circuito Verilog é um buffer. Trata-se de um exemplo com apenas uma entrada e apenas uma saída. Um buffer é um circuito cuja saída reproduz (o valor é igual) a seu sinal de entrada. A palavra chave para descrever um buffer é **buf**, e a sintaxe de um comando verilog é **buf(s, a);** significando que “existe um buffer cujo sinal de saída é **s** e cujo sinal de entrada é **a**”. O exemplo de um buffer descrito em Verilog, acompanhado de seu arquivo **.pinout** compatível, é apresentado na Figura 7. Uma vez que estes textos estejam digitados nas áreas do Verilog e do pinout, é possível compilar o circuito na placa virtual Pitanga clicando em uma das duas opções de compilação indicadas por retângulos vermelhos na Figura 8. Este exemplo irá se comportar de modo que se a chave **sw0** estiver para baixo (desligada) o **led0** estará desligado; enquanto se a chave **sw0** estiver para cima (ligada) o **led0** estará ligado.

```
Hardware Description / Descrição do Hard Menu ▾
1  module myBuffer(a, s);
2      input      a;
3      output     s;
4      buf(s, a);
5  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      a = sw0;
2      s = led0;
```

Figura 7: Exemplo de um buffer incluindo arquivo Verilog e arquivo *.pinout*.

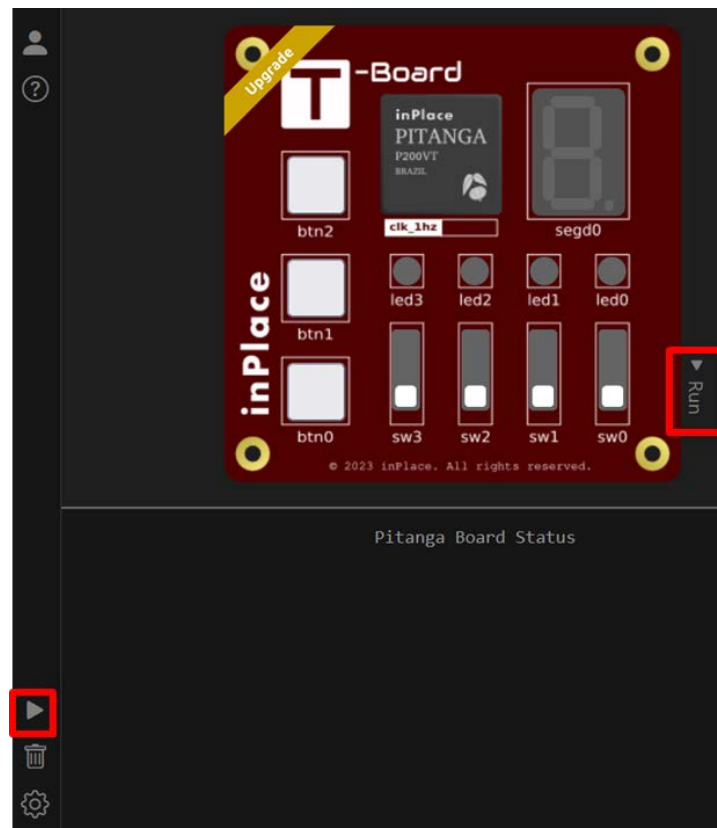


Figura 8: Duas possibilidades (indicadas pelos retângulos vermelhos) de iniciar a compilação de um conjunto composto de um arquivo Verilog e arquivo *.pinout*.

4.2. Segundo exemplo: uma entrada e duas saídas

Outro exemplo bastante simples é um circuito com um buffer e um inversor ligados na mesma chave de entrada. Um possível conjunto de código Verilog e arquivo *.pinout* para este circuito é apresentado na Figura 9.

```
Hardware Description / Descrição do Menu ▾
1  module twoLights(a, s0, s3);
2      input    a;
3      output   s0, s3;
4      buf(s0, a);
5      not(s3, a);
6  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      a  = sw0;
2      s0 = led0;
3      s3 = led3;
```

Figura 9: Exemplo de um buffer e um inversor incluindo arquivo Verilog e arquivo *.pinout*.

O circuito se comporta de tal forma que quando a chave **sw0** está desligada ($a=0$), o **led0** estará apagado ($s0=0$) e o **led3** estará aceso ($s3=1$). Este comportamento pode ser visto na Figura 10. De modo similar, quando a chave **sw0** está ligada ($a=1$), o **led0** estará aceso ($s0=1$) e o **led3** estará apagado ($s3=0$). Este comportamento pode ser visto na Figura 11.

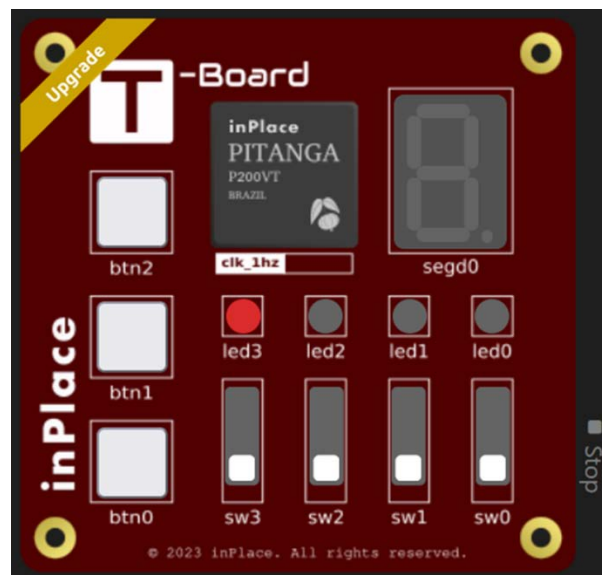


Figura 10: O circuito descrito na Figura 9 com a chave **sw0** desligada.

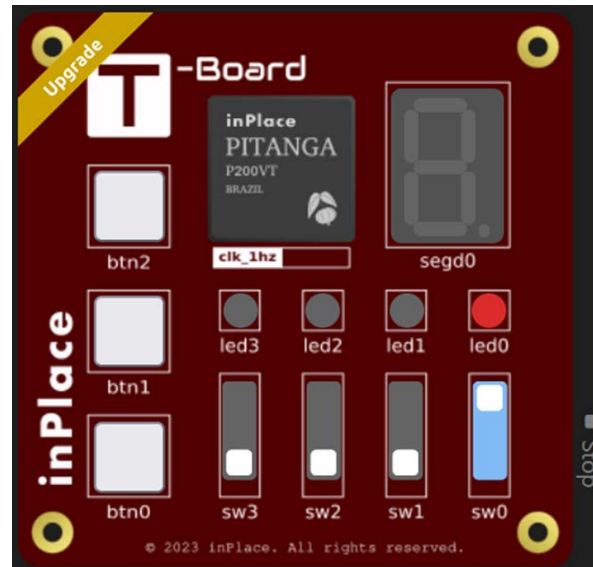


Figura 11: O circuito descrito na Figura 9 com a chave **sw0** ligada.

4.3. Terceiro exemplo: duas entradas e duas saídas

Um exemplo clássico de um circuito com duas entradas e duas saídas é um meio somador. A figura 12 apresenta o Verilog e o `.pinout` de um meio somador. O circuito meio somador faz a contagem de quantas entradas do circuito estão ligadas (ou seja, apresentam o valor 1). A título de exemplo, a Figura 13 apresenta o meio somador com duas entradas ligadas, mostrando o número binário dois (10) na saída já que o **led1** está ligado (**cout=1**) e o **led0** está desligado (**sum=0**).

```
Hardware Description / Descrição do Menu ▾
1  module halfAdder(a, b, cout, sum);
2      input      a, b;
3      output     cout, sum;
4      and(cout, a, b);
5      xor(sum, a, b);
6  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      a      = sw1;
2      b      = sw0;
3      cout   = led1;
4      sum    = led0;
```

Figura 12: Exemplo de um meio somador incluindo arquivo Verilog e arquivo `.pinout`.

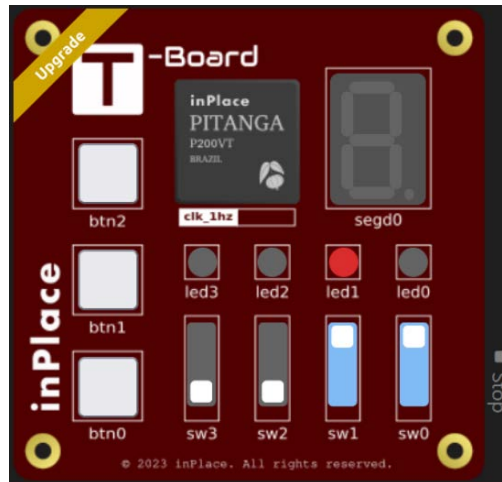


Figura 13: Exemplo de um meio somador com duas entradas ligadas, mostrando o número binário dois (10) na saída já que o **led1** está ligado (**cout=1**) e o **led0** está desligado (**sum=0**).

4.4. Quarto exemplo: três entradas e duas saídas

Um exemplo clássico de um circuito com três entradas e duas saídas é um somador completo. O esquemático de um somador completo é mostrado na Figura 14. Note que apesar de existirem cinco fios entrando no lado esquerdo do esquemático, o circuito tem apenas três variáveis de entrada, pois **a** e **b** estão repetidos. Assim, o circuito tem três variáveis de entrada: **a**, **b** e **cin**. No lado direito do esquemático são mostradas as duas saídas: **cout** e **sum**. Além dos sinais de entrada e saída, o esquemático apresenta três fios internos que não são nem entradas nem saídas: **x**, **y** e **z**. O esquemático é composto de cinco portas lógicas no total: duas portas lógicas tipo *xor*, duas portas lógicas do tipo *and* e uma porta lógica do tipo *or*.

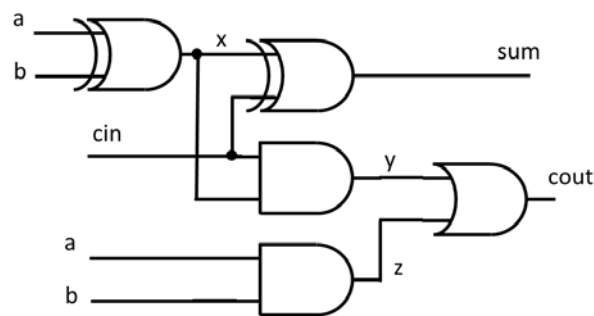


Figura 14: Exemplo do esquemático de um somador completo.

A figura 15 apresenta o Verilog e o `.pinout` de um somador completo feito a partir do esquemático apresentado na Figura 14. Para facilitar o entendimento, a Figura 16 repete o esquemático do somador completo com as portas lógicas numeradas de acordo com a linha em que aparecem na descrição Verilog mostrada à direita. Note que apenas as cinco linhas onde as portas lógicas são descritas é

mostrada na Figura 16. Também é importante perceber que a numeração das linhas é uma facilidade do editor de texto e não é parte da descrição Verilog.

```
Hardware Description / Descrição do Hardware: Menu ▾
1  module fullAdder(a, b, cin, sum, cout);
2  input a, b, cin;
3  output sum, cout;
4  wire x, y, z;
5  xor(x, a, b);
6  xor(sum, x, cin);
7  and(y, cin, x);
8  and(z, a, b);
9  or(cout, y, z);
10 endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1  a = sw2;
2  b = sw1;
3  cin = sw0;
4  sum = led0;
5  cout = led1;
```

Figura 15: Exemplo de um somador completo incluindo arquivo Verilog e arquivo *.pinout*.

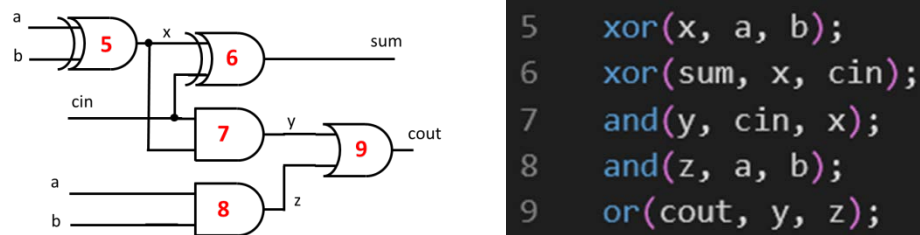


Figura 16: Esquemático do somador completo com as portas lógicas numeradas de acordo com a linha em que aparecem na descrição Verilog mostrada à direita.

O circuito somador completo faz a contagem de quantas entradas do circuito estão ligadas (ou seja, apresentam o valor 1). A título de exemplo, a Figura 17 apresenta o meio somador com três entradas ligadas, mostrando o número binário três (10) na saída já que o *led1* está ligado (*cout=1*) e o *led0* também está ligado (*sum=1*).

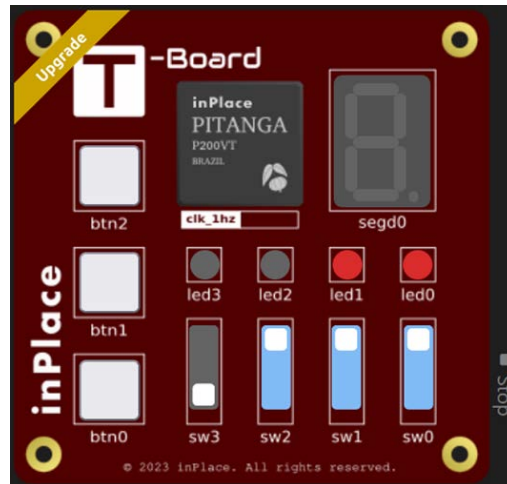


Figura 17: Exemplo de um somador completo com três entradas ligadas, mostrando o número binário três (11) na saída já que o **led1** está ligado (**cout=1**) e o **led0** está ligado (**sum=1**).

4.5. Quinto exemplo: somador hierárquico de dois bits

Todos os exemplos anteriores eram feitos de apenas um módulo. Nesta seção veremos um exemplo composto de mais de um módulo. Para isto vamos utilizar o módulo somador completo e o módulo meio somador, descritos anteriormente, para fazer um somador de duas palavras de dois bits. O esquemático do somador de duas palavras de dois bits é mostrado na Figura 18. O circuito tem quatro bits de entrada divididos em dois vetores de dois bits (vetor de entrada **a** e vetor de entrada **b**) e três bits de saída que são agrupados no vetor de saída **s**. O esquemático da figura 18 será usado como base para a criação de um Verilog hierárquico instanciando um somador completo e um meio somador.

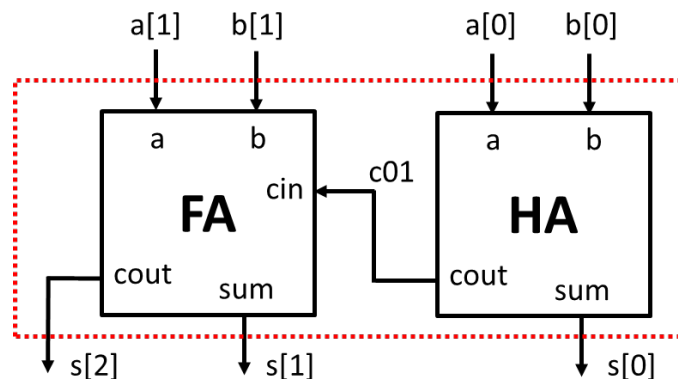


Figura 18: Esquemático de um somador de duas palavras de dois bits composto de um somador completo (FA) e um meio somador (HA).

```
Hardware Description / Descrição do Hardware: Menu ▼
1  module twoBitAdder(a, b, s);
2  input [1:0] a, b;
3  output [2:0] s;
4  wire c01;
5  fullAdder FA(.a(a[1]), .b(b[1]), .cin(c01),
6  |           | .sum(s[1]), .cout(s[2]));
7  halfAdder HA(.a(a[0]), .b(b[0]),
8  |           | .cout(c01), .sum(s[0]));
9  endmodule
10
11 module fullAdder(a, b, cin, sum, cout);
12 input a, b, cin;
13 output sum, cout;
14 wire x, y, z;
15 xor(x, a, b);
16 xor(sum, x, cin);
17 and(y, cin, x);
18 and(z, a, b);
19 or(cout, y, z);
20 endmodule
21
22 module halfAdder(a, b, cout, sum);
23 input a, b;
24 output cout, sum;
25 and(cout, a, b);
26 xor(sum, a, b);
27 endmodule
```

Figura 19: Código Verilog completo do somador de dois bits.

O código completo do Verilog hierárquico instanciando um somador completo e um meio somador é mostrado na Figura 19. A seguir comentamos o código linha a linha, com algumas discussões agrupadas por várias linhas. A linha 1 define o módulo *twoBitAdder* com pinos *a*, *b* e *s*. A linha 2 define os pinos *a* e *b* como entradas que são vetores de dois bits. A linha 3 define o pino *s* como uma saída que é um vetor de três bits. As linhas 5 e 6 fazem a instância de um módulo tipo *fullAdder* (definido na linha 11) cujo nome da instância é *FA*. A sintaxe *.a(a[1])* significa que o pino *a* do *fullAdder* *FA* está conectado ao sinal *a[1]* do módulo onde o *fullAdder* é instanciado, no caso módulo *twoBitAdder*. Esta conexão deve ser feita para todos os pinos do módulo instanciado, no caso do *fullAdder*: *a*, *b*, *cin*, *sum* e *cout*. O texto Verilog para realização das conexões da instância do *fullAdder* *FA* de acordo com o esquemático da Figura 18 é apresentado na tabela 2. As linhas 7 e 8 fazem a instância de um módulo tipo *halfAdder* (definido na linha 22) cujo nome da instância é *HA*. O texto Verilog para realização das conexões da instância do *halfAdder* *HA* de acordo com o esquemático da Figura 18 é apresentado na tabela 2.

O arquivo *.pinout* correspondente ao módulo *twoBitAdder* é apresentado na Figura 20. Note que o assinalamento dos pinos é feito bit a bit. O vetor de entrada *a* está nas chaves *sw3* e *sw2*. O vetor de entrada *b* está nas chaves *sw1* e *sw0*. O vetor de saída *s* é mostrado nos leds *led2*, *led1* e *led0*.

Tabela 2: Descrição da sintaxe das conexões nas instâncias do módulo somador completo (*fullAdder*) e do módulo meio somador (*halfAdder*) no circuito somador de dois bits (*twoBitAdder*).

Instância	Circuito	Sintaxe	Comentário
FA	fullAdder	.a(a[1])	A entrada a do fullAdder está ligada na entrada a[1] do twoBitAdder
FA	fullAdder	.b(b[1])	A entrada b do fullAdder está ligada na entrada b[1] do twoBitAdder
FA	fullAdder	.cin(c01)	A entrada cin do fullAdder está ligada no fio c01 do twoBitAdder
FA	fullAdder	.sum(s[1])	A saída sum do fullAdder está ligada na saída s[1] do twoBitAdder
FA	fullAdder	.cout(s[2])	A saída cout do fullAdder está ligada na saída s[2] do twoBitAdder
HA	halfAdder	.a(a[0])	A entrada a do halfAdder está ligada na entrada a[0] do twoBitAdder
HA	halfAdder	.b(b[0])	A entrada b do halfAdder está ligada na entrada b[0] do twoBitAdder
HA	halfAdder	.cout(c01)	A saída cout do halfAdder está ligada no fio c01 do twoBitAdder
HA	halfAdder	.sum(s[0])	A saída sum do halfAdder está ligada na saída s[0] do twoBitAdder

```

Pinout / Pinagem: pitanga.pinout
1      a[1] = sw3;
2      a[0] = sw2;
3      b[1] = sw1;
4      b[0] = sw0;
5      s[2] = led2;
6      s[1] = led1;
7      s[0] = led0;

```

Figura 20: Arquivo *.pinout* do somador de dois bits.

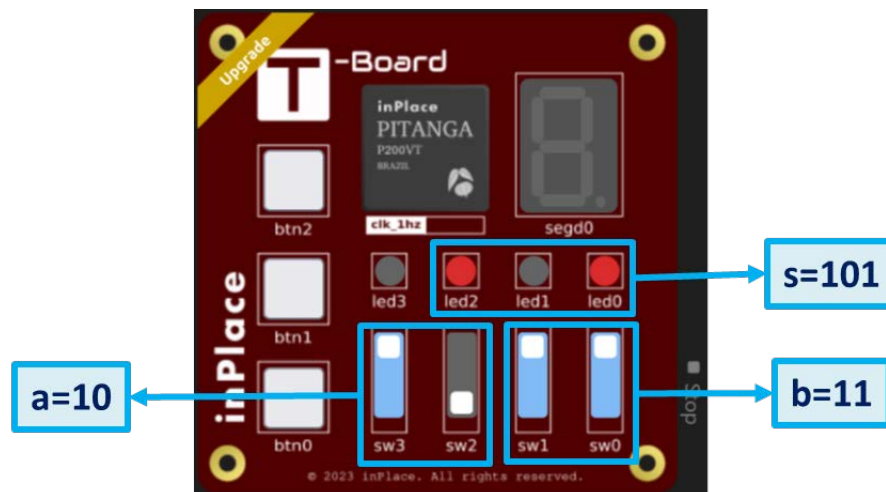


Figura 21: Somador de dois bits executando a soma $2+3=5$, que em binário é $10+11=101$.

A Figura 21 mostra como o módulo *twoBitAdder* se comporta na placa virtual, considerando a configuração de arquivo *.pinout* apresentada na Figura 20. A Figura 21 mostra a soma $2+3=5$, que em binário é $10+11=101$.

5. Entradas e saídas na placa

As entradas de uma placa permitem controlar os valores lógicos nos pinos de entrada do módulo de hardware inserido na placa. A placa virtual Pitanga possui entradas na forma de botões, chaves e relógios.

As saídas de uma placa permitem observar os valores lógicos nos pinos de saída do módulo de hardware colocado na placa. A placa virtual Pitanga tem saídas na forma de leds e displays de sete segmentos com ponto decimal.

É importante saber usar as opções de entrada e saída de modo adequado para facilitar a interação com o módulo de hardware presente na placa. As diferentes opções de entrada e saída, bem como seu uso e funcionamento, são discutidas nas seções a seguir.

5.1. Botões e como usá-los

Botões de apertar são dispositivos comumente utilizados em placas eletrônicas. Seu funcionamento é tal que quando estão soltos geram um valor lógico (normalmente zero, mas pode ser um) e quando estão apertados geram o valor complementar (normalmente um, mas pode ser zero). Botões são usados quando se quer fazer uma sequência de mudanças rápidas em um ou poucos sinais, o que pode ser feito apertando e soltando os botões em uma sequência rápida de movimentos. Botões não são adequados para entrar com um vetor com um valor preciso de um vetor de bits, pois é difícil controlar o estado (solto/apertado) de vários botões ao mesmo tempo.

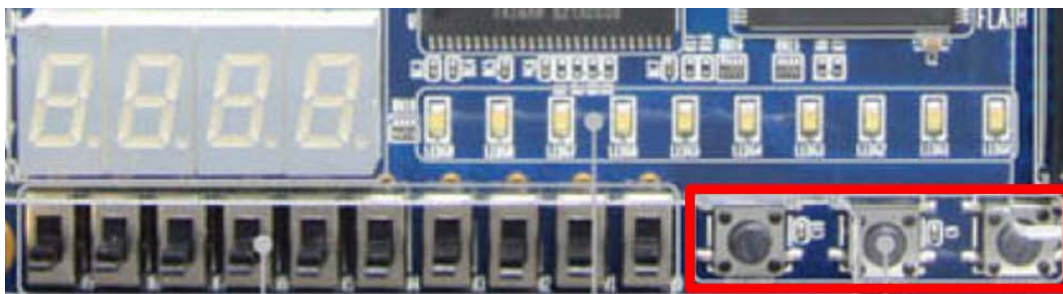


Figura 22: Botões físicos de apertar em uma placa FPGA.

A Figura 22 ilustra os botões físicos de apertar em uma placa FPGA. No caso da placa virtual Pitanga os botões são virtuais e devem ser clicados com o mouse. A Figura 23 ilustra a área dos botões na placa virtual *T-board*, que apresenta três botões denominados **btn2**, **btn1** e **btn0**, conforme indicado na placa. O botão de apertar **btn1** está apertado ($a=1$, portanto), conforme indicado pela flecha representando a posição do mouse. O circuito é o mostrado na Figura 24 com a entrada **a** ligada no botão **btn1**, conforme a linha 1 do *.pinout*. Isto faz com que o **led0** fique aceso, já que $s0=1$.

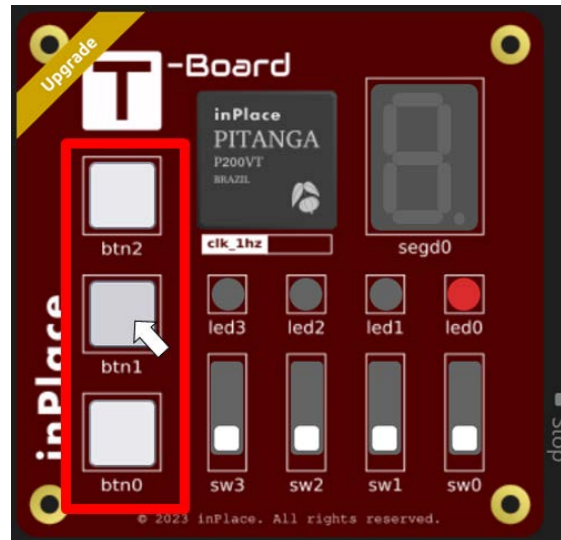


Figura 23: Botões virtuais de apertar (com o mouse) em na placa virtual Pitanga. Placa exibindo o circuito da Figura 24 com o botão de apertar **btn1** apertado.

```
Hardware Description / Descrição do Hardware: Menu ▾
1  module twoLights(a, s0, s3);
2      input    a;
3      output   s0, s3;
4      buf(s0, a);
5      not(s3, a);
6  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1  a = btn1;
2  s0 = led0;
3  s3 = led3;
```

Figura 24: Exemplo de um buffer e um inversor, repetindo o Verilog conforme a Figura 9, mas com o arquivo *.pinout* ligando a entrada **a** no botão de apertar **btn1**.

5.2. Chaves e como usá-las

Além dos botões de apertar, chaves de liga e desliga são dispositivos comumente utilizados em placas eletrônicas. Seu funcionamento é tal que quando estão na posição desligada (posição “para baixo”) geram um valor lógico (normalmente zero, mas pode ser um) e quando estão na posição ligada geram o valor complementar (normalmente um, mas pode ser zero). Chaves de liga/desliga são adequadas para entrar com um vetor com um valor preciso de um vetor de bits, pois é fácil controlar o estado (ligada/desligada) de várias chaves ao mesmo tempo. Chaves de liga/desliga não são indicadas quando

se quer fazer uma sequência de mudanças rápidas em um sinal, pois é difícil mudar o estado da chave em uma sequência rápida de movimentos, por questões mecânicas.

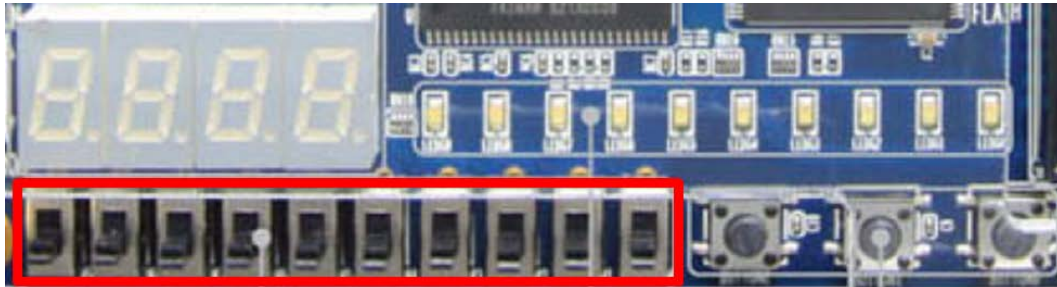


Figura 25: Chaves de liga/desliga em uma placa FPGA.

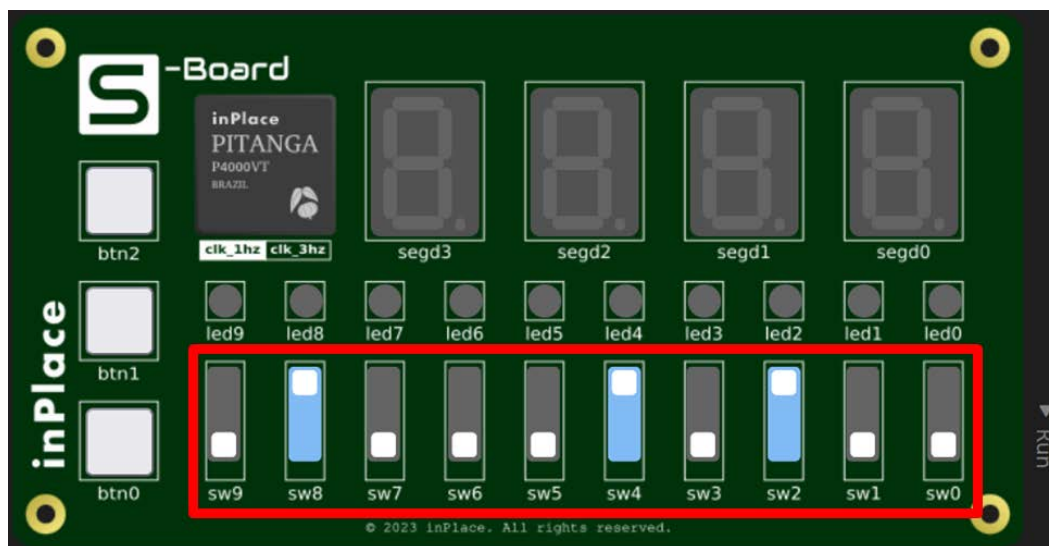


Figura 26: Chaves de liga/desliga em uma placa FPGA.

A Figura 25 ilustra as chaves de liga/desliga físicas em uma placa FPGA. No caso da placa virtual Pitanga as chaves são virtuais e devem ser clicados com o mouse. A Figura 26 ilustra a área dos botões na placa virtual *S-board* (versão student), que apresenta dez chaves de liga/desliga denominadas de **sw9** até **sw0**, conforme indicado na placa. As chaves **sw8**, **sw4** e **sw2** estão ligadas (fornecendo o valor lógico 1 ao circuito), enquanto as demais chaves estão desligadas (fornecendo o valor lógico 0 ao circuito). Vários arquivos *.pinout* usando as chaves de liga/desliga foram apresentados anteriormente, então seu uso deve estar suficientemente claro.

5.3. Relógios e como usá-los

Um sinal de relógio em uma placa de FPGA ou em uma placa virtual é um sinal que corresponde a uma onda quadrada com uma frequência fixa e determinada. No caso da placa Pitanga, os relógios disponíveis estão indicados através de nomes na serigrafia da placa. Por exemplo, a Figura 27 ilustra que a placa *S-Board* oferece os sinais de relógio *clk_1hz* e *clk_3hz*.

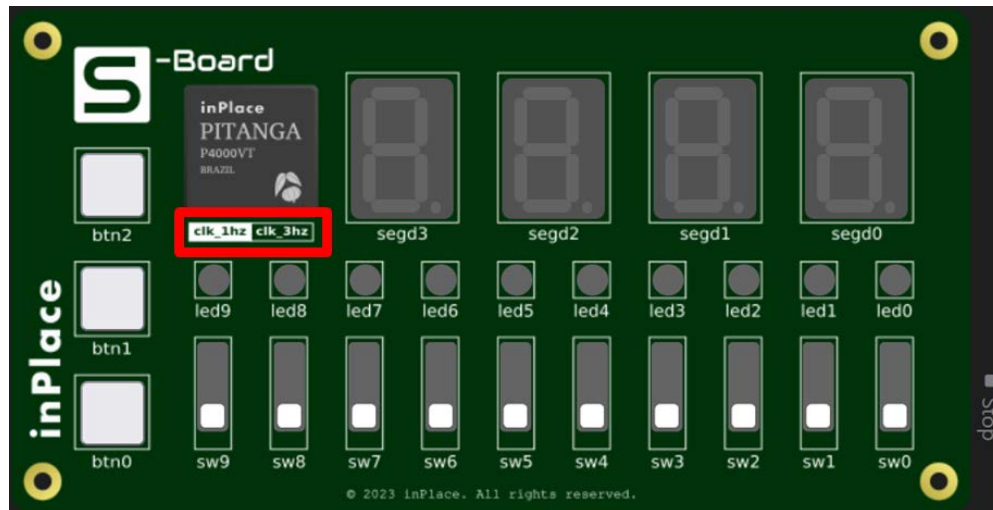


Figura 27: Sinais de relógio disponíveis na placa S-board.

```
Hardware Description / Descrição do Hardware Menu ▾
1  module oneClockTwoLeds1Hz(ck, 10, 13);
2      input    ck;
3      output   10, 13;
4      buf(13, ck);
5      not(10, ck);
6  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      ck = clk_1hz;
2      10 = led0;
3      13 = led3;
```

Figura 28: Circuito com entrada *ck* ligada no pino de relógio *clk_1hz*.

O circuito mostrado na Figura 28 faz os pinos de saída *led0* e *led3* piscarem alternadamente uma vez por segundo, devido a frequência de relógio de 1Hz do pino *clk_1hz*. Já o circuito mostrado na Figura 29 faz os pinos de saída *led0* e *led9* piscarem alternadamente três vezes por segundo, devido a frequência de relógio de 3Hz do pino *clk_3hz*. Note que ao contrário dos botões de apertar e das chaves de liga/desliga, as mudanças de valor dos pinos de relógio ocorrem automaticamente sem necessidade de ação do usuário.

```
Hardware Description / Descrição do Hardware Menu ▾
1  module oneClockTwoLeds3Hz(ck, l0, l9);
2      input      ck;
3      output     l0, l9;
4      buf(l9, ck);
5      not(l0, ck);
6  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      ck = clk_3hz;
2      l0 = led0;
3      l9 = led9;
```

Figura 29: Circuito com entrada **ck** ligada no pino de relógio **clk_3hz**.

5.4. Entradas constantes e como usá-las

Em algumas oportunidades é importante saber usar constantes na descrição de circuitos. O uso de constantes pode ser feito através de pinos com valor constante no arquivo `.pinout`, ou através de constantes definidas na própria linguagem Verilog. A seguir veremos quatro formas diferentes de manter um led sempre aceso.

<pre>Hardware Description / Descrição do Hardware Menu ▾ 1 module sempreAcesoGND(a, s); 2 input a; 3 output s; 4 not(s, a); 5 endmodule Pinout / Pinagem: pitanga.pinout Menu ▾ 1 a = GND; 2 s = led0;</pre>	<pre>Hardware Description / Descrição do Hardware Menu ▾ 1 module sempreAcesoVCC(a, s); 2 input a; 3 output s; 4 buf(s, a); 5 endmodule Pinout / Pinagem: pitanga.pinout Menu ▾ 1 a = VCC; 2 s = led0;</pre>
---	---

Figura 30: Dois circuitos que mantêm o **led0** sempre aceso pelo uso de constantes no arquivo `.pinout`.

A figura 30 mostra dois circuitos bastante similares, os dois circuitos tem uma entrada e uma saída. O circuito da esquerda é um inversor, enquanto o circuito da direita é um buffer. No circuito da esquerda, o arquivo `.pinout` liga a entrada **a** do módulo Verilog no pino **GND** da placa virtual Pitanga, que corresponde a constante zero. Desta forma a saída **s**, através do inversor, está sempre em 1, de modo que o **led0** permanece aceso. Já no caso do circuito da direita o arquivo `.pinout` liga a entrada **a** do módulo Verilog no pino **VCC** da placa virtual Pitanga, que corresponde a constante um. Desta forma a saída **s**, através do buffer, está sempre em 1, de modo que o **led0** permanece aceso.


```
Hardware Description / Descrição Menu ▾  
1 module sempreAceso1b0(s);  
2     output    s;  
3     not(s, 1'b0);  
4 endmodule  
  
Pinout / Pinagem: pitanga.pinoi Menu ▾  
1     s = led0;
```

```
Hardware Description / Descrição Menu ▾  
1 module sempreAceso1b1(s);  
2     output    s;  
3     buf(s, 1'b1);  
4 endmodule  
  
Pinout / Pinagem: pitanga.pinoi Menu ▾  
1     s = led0;
```

Figura 31: Dois circuitos que mantêm o **led0** sempre aceso pelo uso de constantes Verilog.

A figura 31 mostra dois circuitos bastante similares, os dois circuitos tem somente um pino saída, sem entradas. O circuito da esquerda é um inversor, enquanto o circuito da direita é um buffer. No circuito da esquerda, a entrada do inversor é **1'b0**, que corresponde a constante zero em Verilog. Desta forma a saída **s**, através do inversor, está sempre em 1, de modo que o **led0** permanece aceso. Já no caso do circuito da direita a entrada do buffer é **1'b1**, que corresponde a constante um em Verilog. Desta forma a saída **s**, através do buffer, está sempre em 1, de modo que o **led0** permanece aceso.

5.5. Leds e como usá-los

Leds são diodos emissores de luz. A sigla LED vem de *Ligh-Emitting Diode*, em inglês. Na prática é um ponto de luz que pode ser aceso ou apagado dependendo do valor lógico aplicado. A Figura 32 mostra leds físicos em uma placa FPGA. No caso da placa Pitanga os leds são virtuais, como mostrado na Figura 33, que ilustra a placa *S-Board*.

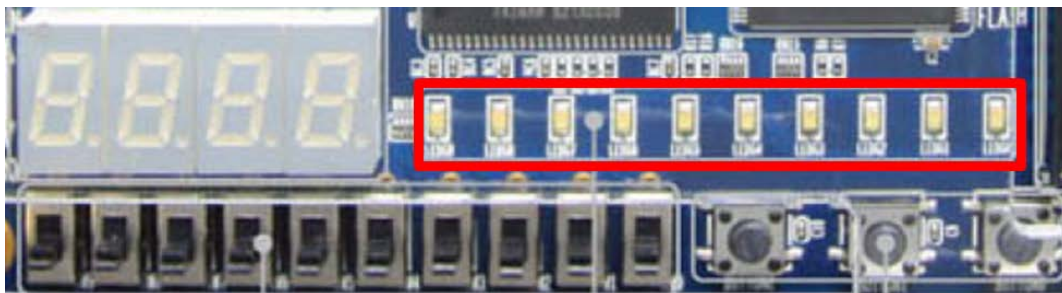


Figura 32: Leds em uma placa FPGA.

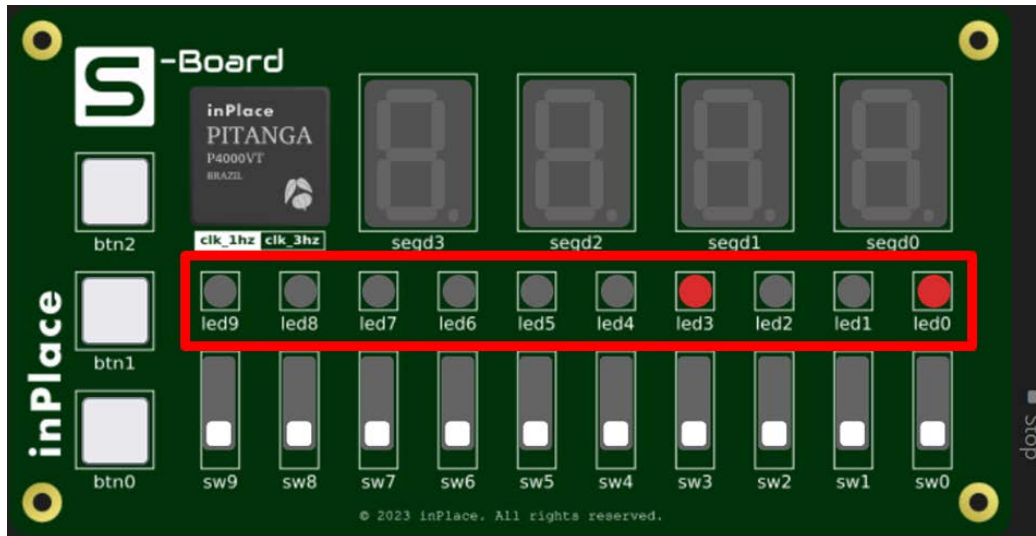


Figura 33: Leds na placa virtual Pitanga S-Board.

A Figura 33 ilustra a área dos leds na placa virtual *S-board* (versão student), que apresenta dez leds denominados de *led9* até *led0*, conforme indicado na placa. Os leds *led3* e *led0* estão ligados (recebendo o valor lógico 1 do circuito), enquanto os demais leds estão desligados (fornecendo o valor lógico 0 ao circuito) ou fora de uso. Vários arquivos *.pinout* usando os leds foram apresentados anteriormente, então seu uso deve estar suficientemente claro.

5.6. Displays e como usá-los

Displays de sete segmentos também são leds, apenas tem formas alongadas e uma disposição específica que permite formar letras e números quando estão ligados/desligados adequadamente em seu conjunto. A figura 34 mostra displays de sete segmentos físicos em uma placa FPGA. De modo similar, a Figura 35 mostra os quatro displays de sete segmentos da placa Pitanga *S-Board*. Estes displays são chamados *seqd3*, *seqd2*, *seqd1* e *seqd0*. O display *seqd0* mostra o número 3, enquanto os outros estão desligados.



Figura 34: Displays de sete segmentos em uma placa FPGA.

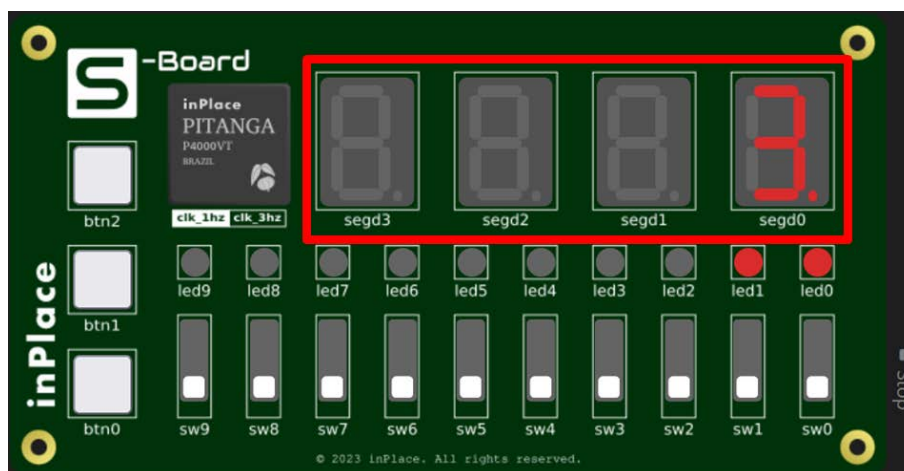


Figura 35: Displays de sete segmentos na placa Pitanga S-Board.

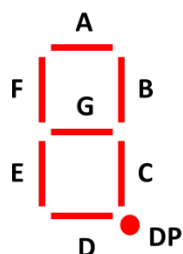


Figura 36: Nomes padrão para leds que compõem os segmentos de um display de sete segmentos.

Tabela 3: Nomes individuais dos segmentos dos quatro displays da placa Pitanga para uso no arquivo *.pinout*.

Segmento	Display			
	segd3	segd2	segd1	segd0
A	segd3.a_on	segd2.a_on	segd1.a_on	segd0.a_on
B	segd3.b_on	segd2.b_on	segd1.b_on	segd0.b_on
C	segd3.c_on	segd2.c_on	segd1.c_on	segd0.c_on
D	segd3.d_on	segd2.d_on	segd1.d_on	segd0.d_on
E	segd3.e_on	segd2.e_on	segd1.e_on	segd0.e_on
F	segd3.f_on	segd2.f_on	segd1.f_on	segd0.f_on
G	segd3.g_on	segd2.g_on	segd1.g_on	segd0.g_on
DP	segd3.dp_on	segd2.dp_on	segd1.dp_on	segd0.dp_on

A figura 36 mostra os nomes padrão adotados para os segmentos de um display de sete segmentos. Como cada segmento é um led individual, é necessário que cada segmento na placa tenha um nome individual para uso no arquivo *.pinout*. A Tabela 3 detalha os nomes individuais dos segmentos dos quatro displays da placa Pitanga para uso no arquivo *.pinout*.

Baseado nos nomes padrão do display de sete segmentos, é possível atribuir valores para os segmentos de modo a formar todos os 16 números hexadecimais de 0 até F. Estes valores são ilustrados visualmente na Figura 37.

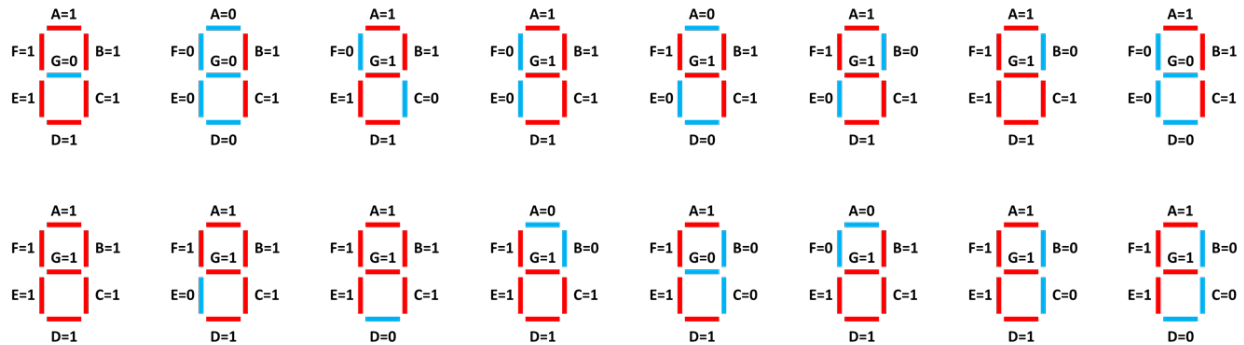


Figura 37: Valores que os segmentos devem assumir para mostrar os 16 números hexadecimais de 0 até F.

Para fazer um circuito de uma entrada que mostre no display de sete segmentos o número 0 quando a entrada é 0, e mostra o numero 1 quando a entrada é 1, são necessárias sete saídas. Isto porque cada um dos segmentos deve receber um valor lógico próprio. A tabela verdade correspondente é mostrada na Tabela 4.

Tabela 4: Tabela verdade para mostrar entrada 0 ou 1 no display de sete segmentos.

in	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0

A Figura 38 apresenta o arquivo Verilog e o arquivo *.pinout* correspondentes ao circuito da tabela verdade mostrada na Tabela 4. As saídas correspondentes aos segmentos **a**, **d**, **e** e **f** são inversores da entrada **in**. As saídas correspondentes aos segmentos **b** e **c** são constantes 1, pois estão sempre ligadas. A saída correspondente ao segmento **g** é uma constante 0. Os nomes no arquivo *.pinout* se referem

```

Hardware Description / Descrição do Hardware: pitanga.v
1  module display01(in, a, b, c, d, e, f, g);
2      input in;
3      output a, b, c, d, e, f, g;
4      not(a, in);
5      buf(b, 1'b1);
6      buf(c, 1'b1);
7      not(d, in);
8      not(e, in);
9      not(f, in);
10     buf(g, 1'b0);
11 endmodule

Pinout / Pinagem: pitanga.pinout
1      in = sw0;
2      g = segd0.g_on;
3      f = segd0.f_on;
4      e = segd0.e_on;
5      d = segd0.d_on;
6      c = segd0.c_on;
7      b = segd0.b_on;
8      a = segd0.a_on;

```

Figura 38: Módulo Verilog e arquivo .pinout para mostrar entrada 0 ou 1 no display de sete segmentos.

Tabela 5: Valores para conversão de binário para sete segmentos, de acordo com a Figura 36.

Hexa	Entrada Binária				Saída sete segmentos						
	i[3]	i[2]	i[1]	i[0]	s[6]	s[5]	s[4]	s[3]	s[2]	s[1]	s[0]
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
A	1	0	1	0	1	1	1	0	1	1	1
B	1	0	1	1	0	0	1	1	1	1	1
C	1	1	0	0	1	0	0	1	1	1	0
D	1	1	0	1	0	1	1	1	1	0	1
E	1	1	1	0	1	0	0	1	1	1	1
F	1	1	1	1	1	0	0	0	1	1	1

Um circuito mais ambicioso pode ser feito para considerar todas as 16 possibilidades mostradas na Figura 36. Esta tabela verdade pode ser feita em forma vetorial e é apresentada na tabela 5. A entrada $i=1101$ gera na saída o vetor $s=0111101$ que mostra o hexadecimal d no display, conforme esperado.

```

Hardware Description / Descrição do Hardware: pitang Menu ▾
1  module display7seg(i, s);
2      input [3:0]    i;
3      output[6:0]    s;
4      assign s =
5          (i == 4'b0000) ? 7'b1111110:
6          (i == 4'b0001) ? 7'b0110000:
7          (i == 4'b0010) ? 7'b1101101:
8          (i == 4'b0011) ? 7'b1111001:
9          (i == 4'b0100) ? 7'b0110011:
10         (i == 4'b0101) ? 7'b1011011:
11         (i == 4'b0110) ? 7'b1011111:
12         (i == 4'b0111) ? 7'b1110000:
13         (i == 4'b1000) ? 7'b1111111:
14         (i == 4'b1001) ? 7'b1111011:
15         (i == 4'b1010) ? 7'b1110111:
16         (i == 4'b1011) ? 7'b0011111:
17         (i == 4'b1100) ? 7'b1001110:
18         (i == 4'b1101) ? 7'b0111101:
19         (i == 4'b1110) ? 7'b1001111:
20         7'b1000111;
21  endmodule

```

Figura 39: Módulo Verilog correspondente à tabela verdade da Tabela 5.

```

Pinout / Pinagem: pitanga.pinout Menu ▾
1      i[3] = sw3;
2      i[2] = sw2;
3      i[1] = sw1;
4      i[0] = sw0;
5      s[0] = segd0.g_on;
6      s[1] = segd0.f_on;
7      s[2] = segd0.e_on;
8      s[3] = segd0.d_on;
9      s[4] = segd0.c_on;
10     s[5] = segd0.b_on;
11     s[6] = segd0.a_on;

```

Figura 40: Arquivo *.pinout* para o módulo Verilog da Figura 39.

A Figura 39 mostra um módulo Verilog correspondente à tabela verdade da Tabela 5. Este módulo foi feito em estilo comportamental usando o comando Verilog *assign* e é quase que uma transcrição literal da tabela 5 fazendo a correspondência entre os vetores de entrada e os vetores de saída do circuito. Por exemplo, a linha 18 mostra que a entrada *i=1101* gera na saída o vetor *s=0111101*. O arquivo *.pinout* para que a saída seja mostrada adequadamente no display de sete segmentos é apresentado na Figura

40. O resultado na placa virtual, correspondente a esta linha na tabela verdade é apresentado na Figura 41. Note que o símbolo hexadecimal **d** é apresentado no display, quando a entrada **i=1101** nas chaves, conforme esperado.

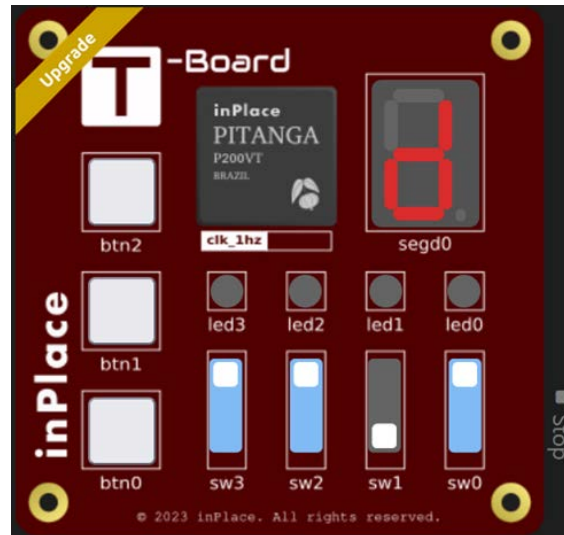


Figura 41: O símbolo hexadecimal **d** é apresentado no display, quando a entrada **i=1101** nas chaves.

Cell	Instances	Cell	Instances	Cell	Instances
AND2	0	NAND2	12	XOR2	0
AND3	0	NAND3	4	XOR3	0
AND4	0	NAND4	0	XOR4	0
OR2	1	NOR2	9	XNOR2	1
OR3	2	NOR3	5	XNOR3	0
OR4	0	NOR4	0	XNOR4	0
BUF	0	INV	4	DFRSE	0
Cells utilization: 38					
Transistor count : 180/200 transistors (90.00 %)					

Figura 42: Relatório de compilação do módulo Verilog da Figura 39.

Finalmente, é importante perceber que apesar de o módulo Verilog da Figura 39 ter sido descrito em nível comportamental, Verilog continua sendo uma linguagem de descrição de hardware. Assim, o relatório de compilação apresentado na Figura 42 descreve quantas portas lógicas e quantos transistores foram necessários para traduzir o Verilog inicial em uma implementação do circuito em hardware.

6. Circuitos sequenciais: Flip-flops

Todos os circuitos apresentados anteriormente eram circuitos sequenciais, ou seja: circuitos sem elementos de memória. Nesta seção veremos alguns exemplos contendo memória, usando Flip-flops definidos de maneira comportamental como o elemento base de memória.

A Figura 43 mostra o esquemático de um circuito divisor de frequência. A entrada é um sinal **clk** cuja frequência é dividida por dois na saída **out**. Na prática o comportamento do circuito na placa é que o led na saída **out** acende e apaga a cada clique no botão de entrada **clk**.

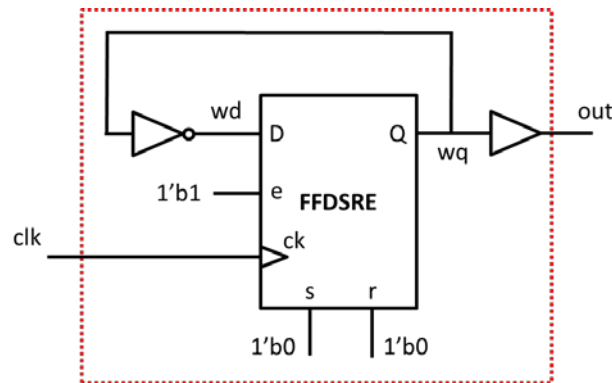


Figura 43: Circuito divisor de frequência composto de um Flip-flop, um inversor e um buffer.

```
Hardware Description / Descrição do Hardware: pit Menu ▼
1  module divFreqByTwo(clk, out);
2      input clk;
3      output out;
4      wire wq, wd;
5      not(wd, wq);
6      FFDSRE ffInstance(.D(wd), .ck(clk),
7          .r(1'b0), .s(1'b0), .e(1'b1), .Q(wq));
8      buf(out, wq);
9  endmodule
10
11 > module FFDSRE(D, ck, r, s, e, Q); ...
23
```

```
Pinout / Pinagem: pitanga.pinout Menu ▼
1  clk = btn0;
2  out = led0;
```

Figura 44: Verilog e *.pinout* correspondentes ao esquemático da Figura 43.

A Figura 44 mostra os arquivos Verilog e *.pinout* correspondentes ao esquemático da Figura 43. As linhas 6 e 7 são uma instância do módulo **FFDSRE** cujo cabeçalho é visível na linha 11. A instancia das linhas 6 e 7 segue a sintaxe de conexões apresentada na Tabela 6.

Tabela 6: Conexões usadas na instância *ffInstance* do módulo **FFDSRE** nas linhas 6 e 7 da Figura 44, conforme o esquemático da Figura 43.

Pino em FFDSRE	D	ck	r	s	e	Q
Pino em <i>divFreqbyTwo</i>	wd	clk	Zero	Zero	Um	wq
Sintaxe da conexão	.D(wd)	.ck(clk)	.r(1'b0)	.s(1'b0)	.e(1'b1)	Q(wq)

O módulo **FFDSRE** é a versão de um código comportamental de um Flip-flop, e pode ser usado como caixa preta para instanciar Flip-flops. Este é o modo aconselhado para garantir compatibilidade com plataformas de outros fabricantes já que o comportamento compatível do Flip-flop é garantido através da descrição comportamental que força um comportamento homogêneo e compatível. Porém o código do módulo **FFDSRE** deve fazer parte do projeto. Por exemplo, na figura 44 usamos o editor de texto para esconder o conteúdo do código entre as linhas 11 e 22, deixando apenas o cabeçalho do módulo visível.

```

Hardware Description / Descrição do Hardwar Menu ▾
1  module FFDSRE(D, ck, r, s, e, Q);
2      input    D, ck, r, s, e;
3      output   reg Q;
4      always@(posedge ck, posedge r) begin
5          if(r)
6              Q <= 1'b0;
7          else if(s)
8              Q <= 1'b1;
9          else if (e)
10             Q <= D;
11      end
12  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      ck = btn0;
2      s  = sw3;
3      e  = sw1;
4      r  = sw2;
5      D  = sw0;
6      Q  = led0;

```

Figura 45: Módulo FFDSRE e arquivo *.pinout* para fazer emulação individual de um Flip-flop.

O conteúdo do módulo **FFDSRE** é mostrado na Figura 45. O módulo **FFDSRE** pode ser usado como caixa preta sem necessidade de ser entendido, mas a seguir explicamos o código. O comportamento de memória é garantido pelo uso de uma variável de saída *Q* do tipo *output reg* (linha 3). O bloco *always* (linhas 4 até 11) define o comportamento do Flip-flop. Conforme a linha 4, o bloco *always* somente é executado quando há mudanças do tipo borda positiva (*posedge*) dos sinais *ck* ou *r*. Ou seja, na prática o reset (*r*) é assíncrono (não depende do sinal de relógio *ck*). O sinal de set (*s*) é síncrono, pois uma

mudança em *s* não causa entrada no bloco *always* (conforme linha 4), e assim o sinal *s* depende de uma mudança no sinal *ck* para entrar no bloco *always* e ser avaliado. O reset é feito na linha 6. O set é feito na linha 8. A cópia do sinal de entrada *D* para a saída *Q* é feita na linha 10, mas somente ocorre se o Flip-flop estiver habilitado (*e=1*). Insistimos que o Flip-flop pode ser usado como caixa preta sem necessidade de entender o código da Figura 45 inicialmente. Isto vai ocorrer mais tarde se você continuar estudando Verilog e descrevendo circuitos digitais. Aproveitamos para insistir que Verilog não é uma linguagem de programação e sim uma linguagem de descrição de hardware. Assim, o código da Figura 45 será traduzido para hardware.

Para entender um pouco melhor o uso do Flip-Flop da Figura 45, vamos apresentar um segundo exemplo. O exemplo é um contador de quatro bits, conforme o esquemático da Figura 46. A primeira observação importante sobre o esquemático da Figura 46 é que existem alguns fios que são barramentos de quatro bits. Na prática é como se fossem cabos, compostos de quatro fios internos, um para cada um dos quatro bits. A estrutura do circuito na Figura 46 é muito semelhante a estrutura do circuito na Figura 43. Porém o circuito da Figura 46 usa módulos de 4 bits. O módulo *reg4* é composto de quatro Flip-flops, como veremos adiante.

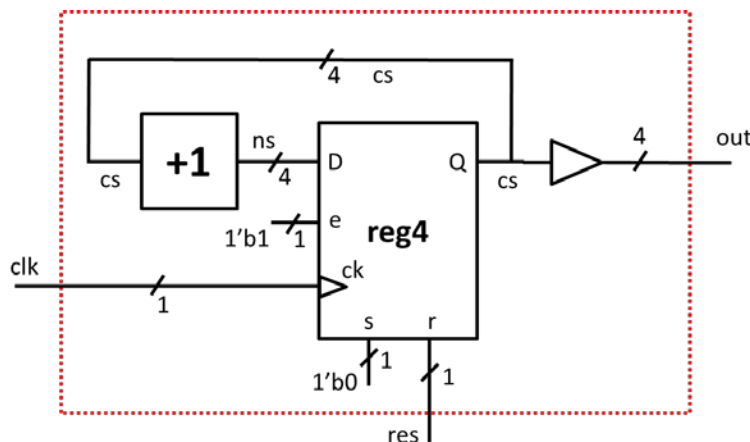


Figura 46: Esquemático de um contador de quatro bits.

O Verilog do módulo principal correspondente ao esquemático do contador de quatro bits mostrado na Figura 46 é apresentado na Figura 47. Note que a saída *out* (linha 3) é do tipo **output [3:0]**, sendo assim uma saída que é um vetor (ou fisicamente um cabo) de quatro bits (fios). Os fios internos *ns* e *cs* (linha 4) são do tipo **wire [3:0]**, podendo também ser entendidos como vetores (ou fisicamente cabos) de quatro bits (fios). O módulo *plusOne* (linha 5) faz a soma de uma unidade no valor atual do registrador (*cs*, de *current state* em inglês) para produzir o próximo valor da contagem (*ns*, de *next state* em inglês). O módulo *reg4* (linhas 6 e 7) é um registrador de quatro bits que é composto de quatro Flip-flops individuais. O módulo *reg4* armazena o estado atual (*cs*) a partir do qual o próximo valor da contagem

(*ns*) é computado somando-se um no módulo *plusOne*. O módulo *buf4* (linha 8) é um buffer de quatro bits usado para isolar o sinal interno *cs* (*current state*) da saída externa *out*.

```
Hardware Description / Descrição do Hardware: p Menu ▾
1  module fourBitCounter(clk, res, out);
2      input clk, res;
3      output [3:0] out;
4      wire [3:0] ns, cs;
5      plusOne po(.in(cs), .out(ns));
6      reg4 regState(.D(ns), .ck(clk),
7          .r(res), .s(1'b0), .e(1'b1), .Q(cs));
8      buf4 bufOut(.in(cs), .out(out));
9  endmodule
```

Figura 47: Código Verilog do módulo principal de um contador de quatro bits em Verilog de acordo com o esquemático da Figura 46.

```
Pinout / Pinagem: pitanga.pinout Menu ▾
1      clk = btn0;
2      res = sw0;
3      out[3] = led3;
4      out[2] = led2;
5      out[1] = led1;
6      out[0] = led0;
```

Figura 48: Arquivo *.pinout* do módulo principal de um contador de quatro bits em Verilog mostrado na Figura 47.

Um possível arquivo *.pinout* para o contador de quatro bits é mostrado na Figura 48. Neste caso o reset é assíncrono e feito através da chave *sw0*. O relógio do circuito está no botão *btn0*, assim a cada clique neste botão o valor armazenado no contador é incrementado de uma unidade. O valor de saída do contador é mostrado em formato binário nos quatro leds no intervalo de *led3* até *led0*.

O código Verilog dos três sub-módulos instanciados no módulo principal é apresentado na Figura 49. O módulo *plusOne* faz a soma de uma unidade, conforme o conteúdo da linha 14. O módulo *reg4* é composto de quatro instâncias de Flip-flop (módulo FFDSRE), conforme as linhas 21 até 24. O módulo *buf4* faz uma passagem do valor da entrada para a saída, através de um assinalamento direto conforme o conteúdo da linha 30.

```

11 module plusOne(in, out);
12   input [3:0] in;
13   output [3:0] out;
14   assign out = in+1;
15 endmodule
16
17 module reg4(D, ck, r, s, e, Q);
18   input [3:0] D;
19   input ck, r, s, e;
20   output [3:0] Q;
21   FFDSRE b3(.D(D[3]), .ck(ck), .r(r), .s(s), .e(e), .Q(Q[3]));
22   FFDSRE b2(.D(D[2]), .ck(ck), .r(r), .s(s), .e(e), .Q(Q[2]));
23   FFDSRE b1(.D(D[1]), .ck(ck), .r(r), .s(s), .e(e), .Q(Q[1]));
24   FFDSRE b0(.D(D[0]), .ck(ck), .r(r), .s(s), .e(e), .Q(Q[0]));
25 endmodule
26
27 module buf4(in, out);
28   input [3:0] in;
29   output [3:0] out;
30   assign out=in;
31 endmodule
32
33 > module FFDSRE(D, ck, r, s, e, Q); ...
45

```

Figura 49: Código Verilog dos módulos instanciados no módulo principal de um contador de quatro bits em Verilog de acordo com o esquemático da Figura 46.

```

Hardware Description / Descrição do Hardware: pitanga.v
1 module fourBitCounterFlat(clk, res, out);
2   input clk, res;
3   output [3:0] out;
4   wire [3:0] ns, cs;
5   assign ns=cs+1;
6   FFDSRE b3(.D(ns[3]), .ck(clk), .r(res), .s(1'b0), .e(1'b1), .Q(cs[3]));
7   FFDSRE b2(.D(ns[2]), .ck(clk), .r(res), .s(1'b0), .e(1'b1), .Q(cs[2]));
8   FFDSRE b1(.D(ns[1]), .ck(clk), .r(res), .s(1'b0), .e(1'b1), .Q(cs[1]));
9   FFDSRE b0(.D(ns[0]), .ck(clk), .r(res), .s(1'b0), .e(1'b1), .Q(cs[0]));
10  assign out=cs;
11 endmodule
12
13 > module FFDSRE(D, ck, r, s, e, Q); ...
25

```

Figura 50: Código Verilog do módulo principal de um contador de quatro bits sem utilizar sub módulos e perdendo a relação direta com o esquemático da Figura 46.

A Figura 50 apresenta uma versão do código da Figura 47 sem utilizar sub-módulos. Neste caso a relação com o esquemático da Figura 46 é perdida e o código sem a hierarquia é muito mais difícil de entender para iniciantes. O código da Figura 50 funciona com o mesmo *.pinout* da Figura 49.

A Figura 51 apresenta o relatório de compilação dos Verilogs mostrados tanto na Figura 47 como na Figura 50 (o número de portas lógicas é o mesmo). Aproveitamos para insistir mais uma vez que Verilog não é uma linguagem de programação e sim uma linguagem de descrição de hardware. Assim, os códigos das Figuras 47 e 50 são traduzidos para hardware, resultando nas 10 portas lógicas listadas na Figura 51, que em seu conjunto contém 200 transistores. Note que como o circuito contém um registrador de 4 bits, quatro Flip-flops do tipo **DDFRSE** da biblioteca de células são utilizados. Estas células da biblioteca chamadas DDRSE não devem ser confundidas com o nome que demos ao nosso Flip-flop comportamental no módulo descrito na Figura 45. A pitanga compila o código Verilog do módulo **FFDRSE** (Figura 45) e mapeia para Flip-flops físicos da biblioteca de células que tem um nome parecido (o Flip-flop físico presente na biblioteca se chama **DDFRSE**, conforme pode ser visto no relatório de compilação da Figura 51). Além dos quatro **DDFRSE** físicos, o circuito usa 1 inversor (**INV**), 1 porta nand de duas entradas (**NAND2**), 1 porta nand de três entradas (**NAND3**), e 3 portas ou-exclusivo de duas entradas (**XNOR2**).

Cell	Instances	Cell	Instances	Cell	Instances
AND2	0	NAND2	1	XOR2	0
AND3	0	NAND3	1	XOR3	0
AND4	0	NAND4	0	XOR4	0
OR2	0	NOR2	0	XNOR2	3
OR3	0	NOR3	0	XNOR3	0
OR4	0	NOR4	0	XNOR4	0
BUF	0	INV	1	DDFRSE	4
Cells utilization: 10					
Transistor count : 200/500 transistors (40.00 %)					

Figura 51: Relatório de compilação dos Verilogs mostrados na Figura 47 e na Figura 50 (o número de portas lógicas é o mesmo).

7. Os planos de assinatura e os tipos de placa

Esta seção apresenta os diferentes planos de assinatura e os tipos de placa disponíveis na plataforma Pitanga. As placas virtuais são oferecidas como serviço de software, a plataforma Pitanga é um serviço de SaaS (*Software as a Service*, software como serviço em inglês) que oferece um serviço de emulação interativa de circuitos descritos em Verilog. O usuário interage com o serviço de emulação via uma placa virtual (existindo também a versão física que está fora do escopo deste manual). A figura 52 ilustra os planos de assinatura de acordo com a página da InPlace. A Figura 53 mostra os três diferentes tipos de placas virtuais em versão *Trial (T-Board)*, *Community (C-Board)* e *Student (S-Board)*. A Figura 54 ilustra a placa física de prateleira (placa padrão, vendida por terceiros) configurável via *Pitanguino*. A seguir apresentamos algumas tabelas para ilustrar as características dos diferentes tipos de planos e placas.

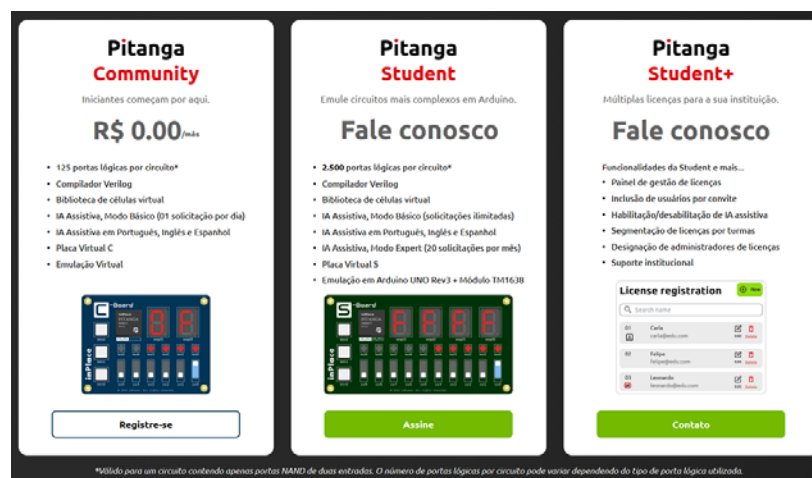


Figura 52: Planos de assinatura segundo a página da InPlace.

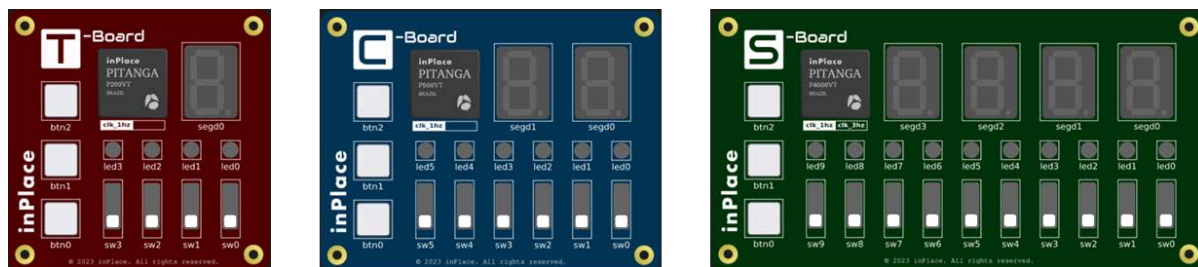


Figura 53: As placas virtuais em versão *Trial (T-Board)*, *Community (C-Board)* e *Student (S-Board)*.

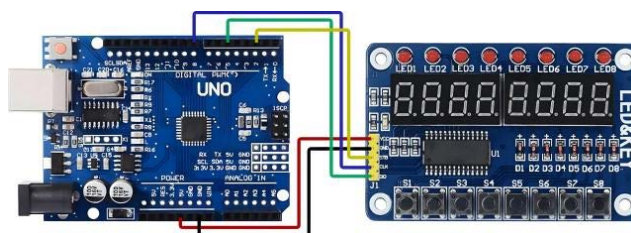


Figura 54: A placa física de prateleira configurável via *Pitanguino* (fora do escopo deste Manual).

A Tabela 7 compara os diferentes tipos de placa virtual e também registra a possibilidade da placa física via agente *Pitanguino*. A última coluna se refere à disponibilidade de acesso a IA Assistiva Neuralis, que é descrita mais adiante neste manual.

Tabela 7: Características de cada placa.

Placa	Tipo	Botões	Chaves	Leds	Displays	Transistores	Neuralis
T-Board	Virtual	3	4	4	1	200	Não
C-Board	Virtual	3	6	6	2	500	Não
S-Board	Virtual	3	10	10	4	10.000	Sim
Pitanguino	Física	8	0	8	4	Depende	Sim

Tabela 8: Planos de assinatura, custo e placas disponíveis.

Assinatura	T-Board	C-Board	S-Board	Pitanguino	Neuralis	Custo	Registro
Trial	Sim	Não	Não	Não	Não	Gratuita	Não
Community	Sim	Sim	Não	Não	Não	Gratuita	Sim
Student	Sim	Sim	Sim	Sim	Sim	Paga	Sim
Student+	Sim	Sim	Sim	Sim	Sim	Paga	Sim

A Tabela 8 compara os diferentes tipos de plano de assinatura, considerando acesso às diferentes placas, acesso à IA assistiva Neuralis e custo do plano. A última coluna se refere à necessidade de registro para uso da placa. No caso do plano *Trial*, placa **T-Board**, o acesso é feito sem necessidade de registro. Os planos *Trial* e *Community* são gratuitos, mas o plano *Community* necessita registro. Os planos *Student* e *Student+* são similares em termos de acesso. O plano *Student* é individual, enquanto o plano *Student+* é voltado a instituições ou professores, permitindo a compra de múltiplas licenças em lote e sua gerência por parte do professor ou responsável.

8. Uso da plataforma Pitanga

Nesta seção descrevemos o uso da plataforma Pitanga. Vários aspectos de uso são descritos e analisados, considerando como fazer uso do sistema uma vez que o circuito já tenha sido descrito e os arquivos Verilog e *.pinout* estejam prontos para uso. As próximas seções descrevem aspectos de uso da plataforma além do Verilog e do *.pinout*.

8.1. Como se logar?

Considerando-se que existem planos que tem como requisito o registro do usuário, é necessário ter acesso ao sistema de registro da plataforma Pitanga. O sistema de registro e login está disponível clicando-se no ícone correspondente no canto superior esquerdo da página, conforme ilustrado pela Figura 55. Uma vez clicando-se no ícone de registro e login, o pop-up mostrado na figura 56 ficará disponível, permitindo as ações de registro e login. Neste pop-up também é possível recuperar a senha de acesso.

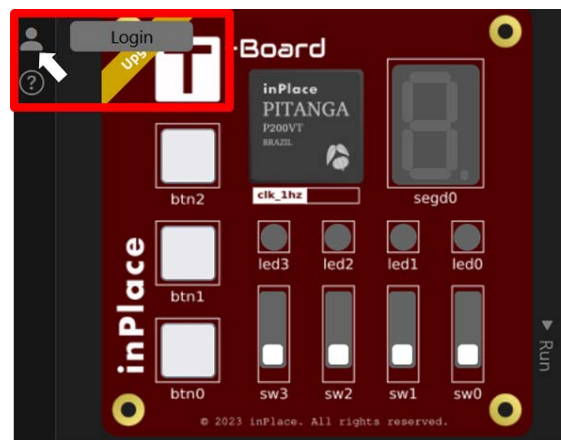


Figura 55: Acesso ao sistema de registro e login.

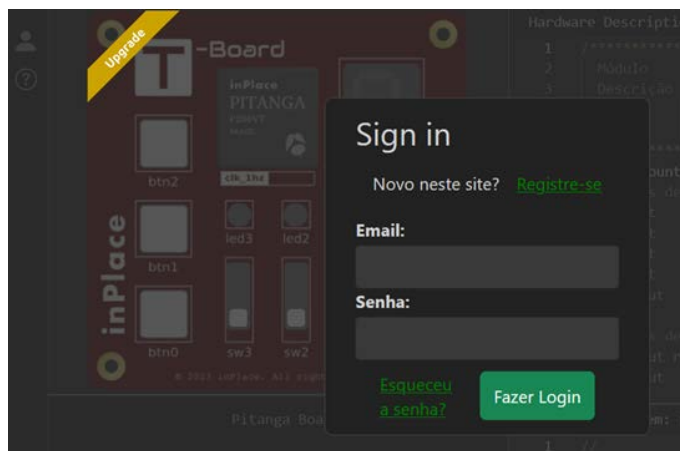


Figura 56: Pop-up de registro e login.

8.2. Como obter ajuda?

Abaixo do ícone de registro e login no canto superior esquerdo da página existe um ícone de ajuda, conforme a Figura 57. Este ícone dá acesso ao pop-up ilustrado na Figura 58, que dá acesso a tutoriais e notícias entre outros.

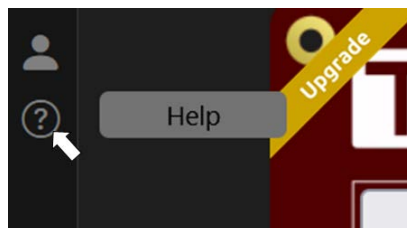


Figura 57: Ícone de ajuda para a Pitanga.



Figura 58: Pop-up com links de ajuda.

8.3. Iniciando e parando a emulação

Existem duas formas distintas de iniciar e de parar a emulação de um circuito na Plataforma Pitanga. Estes botões se localizam na mesma posição e só mudam de função caso a emulação esteja parada ou rodando. A Figura 59 ilustra os dois botões que permitem iniciar a emulação quando ela está inativa. A Figura 60 mostra os dois botões que permitem parar a emulação quando ela está ativa. Nos dois casos a posição dos botões é a mesma. Um dos botões fica na parte inferior esquerda da página, ao lado esquerdo da área de relatório. O outro botão fica no canto inferior direito da placa virtual

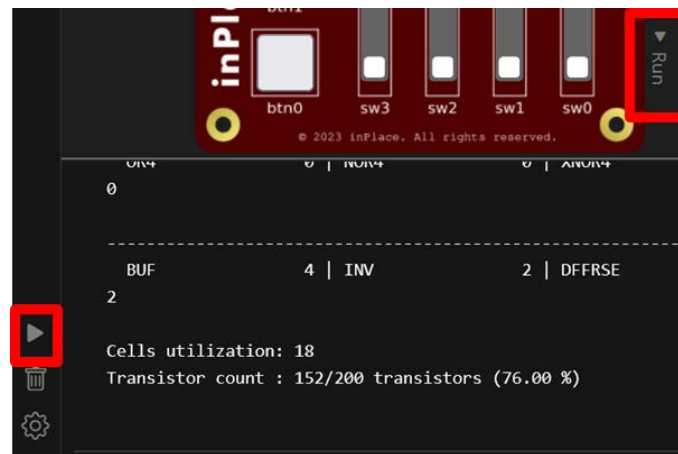


Figura 59: Dois botões para iniciar a emulação.

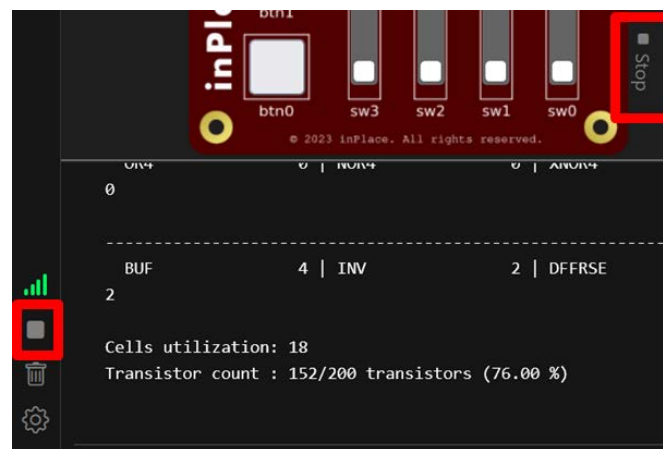


Figura 60: Dois botões que permitem parar a emulação.

8.4. Outros comandos da Pitanga

A parte inferior esquerda da placa virtual apresenta quatro ícones que disponibilizam funcionalidades para a plataforma Pitanga. Estes ícones ficam no lado esquerdo da área de relatório, conforme pode ser visto na Figura 61. A seguir discutimos a funcionalidade disponibilizada por cada um destes ícones.

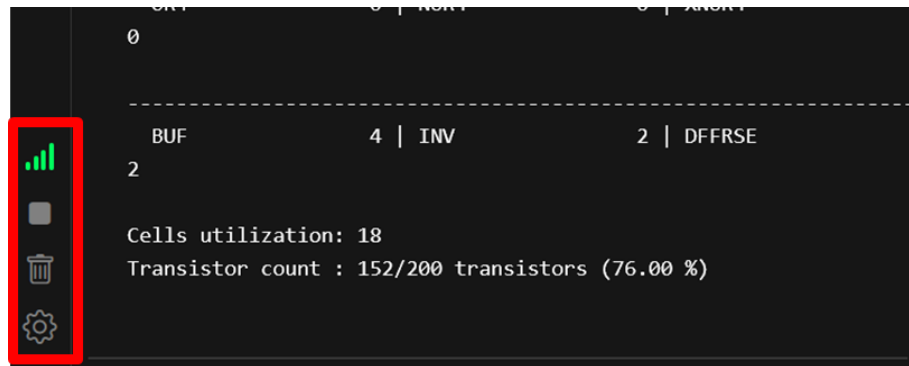


Figura 61: Ícones de funcionalidade no canto inferior esquerdo da página da plataforma Pitanga.

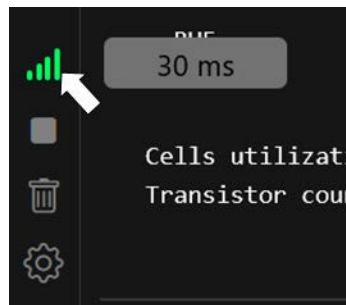


Figura 62: Ícone de conectividade e tempo de resposta.

A Figura 62 mostra o ícone de conectividade e tempo de resposta. Na figura o tempo de resposta do servidor é de 30ms. Este é um tempo que não afeta a qualidade da reação da placa Pitanga, sendo um atraso imperceptível para o usuário. Por isto o ícone está verde. Caso aconteçam tempos de resposta muito lentos que afetam a qualidade da interação o ícone pode ficar amarelo ou vermelho, indicando conexão de baixa qualidade. Abaixo do ícone de conectividade fica o ícone para interromper (ou iniciar) a emulação, já discutido anteriormente.

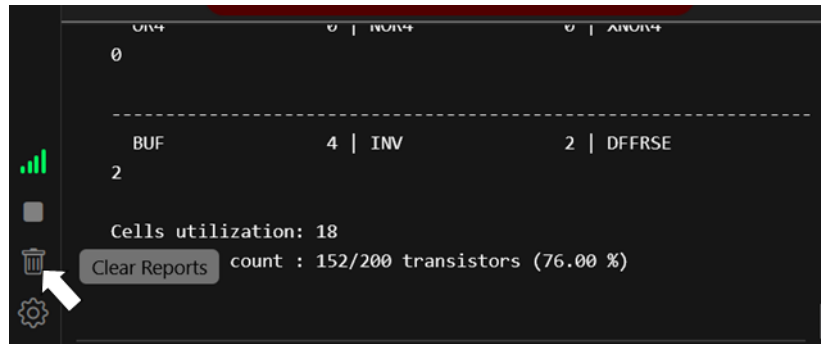


Figura 63: Ícone para apagar o texto na área de relatório.

Abaixo do ícone para interromper (ou iniciar) a emulação, já discutido anteriormente, está localizado o ícone para apagar o texto na área de relatório, conforme mostrado na Figura 63. Esta funcionalidade é útil para dar ao usuário a certeza de que as mensagens de compilação mais recentes são as que estão sendo lidas na área do relatório.

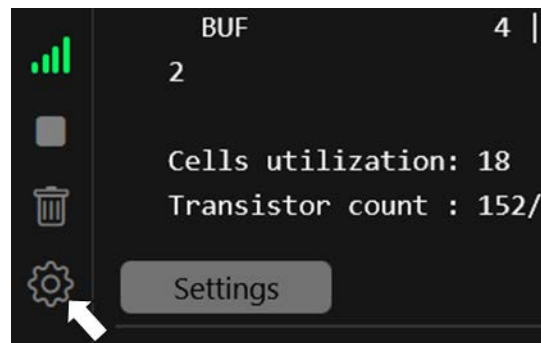


Figura 64: Ícone de configurações.

A Figura 64 ilustra o ícone de configurações da plataforma Pitanga, que está localizado no canto inferior esquerdo da página. Este ícone abre um menu que é apresentado na Figura 65. O menu de configurações permite selecionar entre as placas *student*, *community* e *trial* se estas estiverem habilitadas no plano de assinatura. Outra configuração possível é a escolha do idioma no qual as respostas da IA assistiva Neuralis serão apresentadas. A escolha pode ser feita entre português, inglês e espanhol.

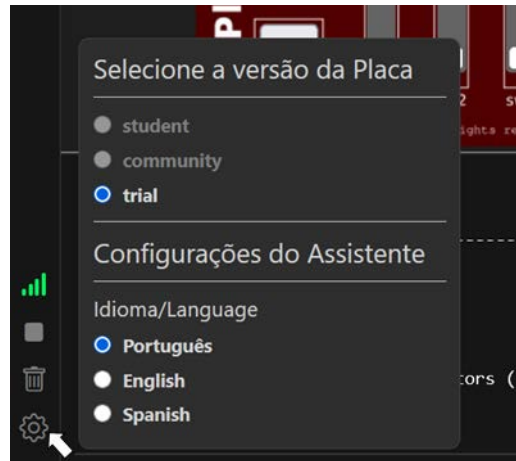


Figura 65: Menu de configurações.

9. IA Assistiva Neuralis

A plataforma Pitanga oferece uma IA assistiva integrada com plataforma para corrigir erros de Verilog que porventura ocorram. O nome desta IA assistiva é Neuralis. Esta seção descreve o uso da IA assistiva Neuralis para corrigir um bug em um exemplo de Verilog.

```
22     always@(posedge clk) begin
23         if(rst)
24             cnt <= 2'b007; // ajusta todos os bits do contador para '0'
25         else if(set)
26             cnt <= 2'b11; // ajusta todos os bits do contador para '1'
27         else if (!cen)
28             cnt <= cnt+1; // incrementa contador cnt em um
29     end
```

Figura 66: Trecho de código Verilog contendo um erro.

A Figura 66 apresenta um trecho de código Verilog contendo um erro. É um erro que passa despercebido, especialmente para iniciantes. Assim, o erro irá aparecer no momento da compilação, quando o usuário iniciante receberá uma mensagem de erro do compilador.

A Figura 67 apresenta uma mensagem de erro obtida ao compilar um trecho de código Verilog contendo um erro, acompanhada de oferta de ajuda da IA assistiva Neuralis. A mensagem de erro do compilador é: *"pitanga.v:24:25: error: expected binary digit cnt <= 2'b007;"*. Trata-se provavelmente de uma mensagem difícil de decifrar para um usuário iniciante. Na plataforma Pitanga esta mensagem vem acompanhada de uma oferta de ajuda que pode ser de dois tipos, conforme visto na Figura 67. Uma das ofertas é para ajuda básica (*Basic*), que pode ser usada quantas vezes for necessária. A outra oferta é

para auxílio especialista (Expert) que tem um contador de número de vezes que pode ser usada diariamente, por ser uma IA que tem um custo maior. Um total de 20 usos ainda estão disponíveis no caso da Figura 67.

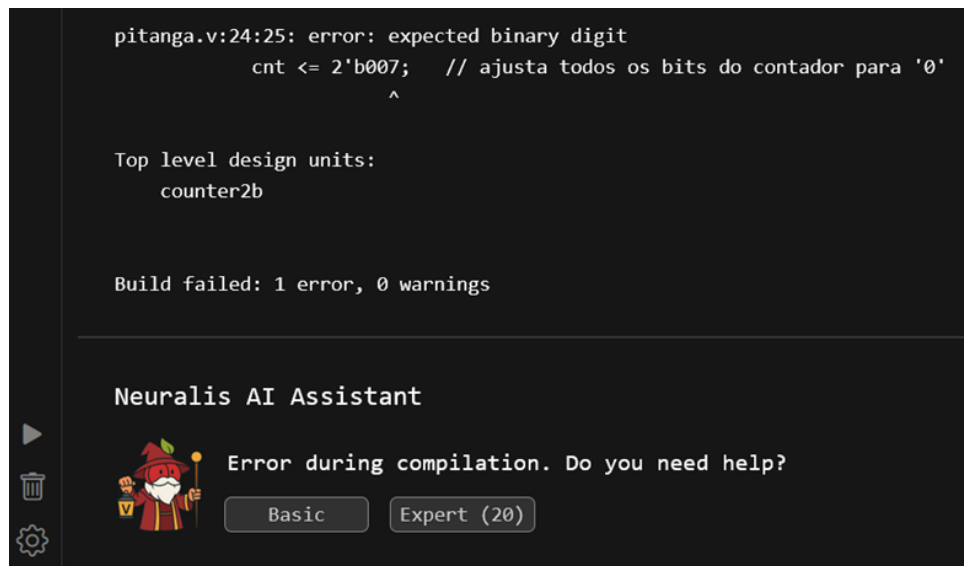


Figura 67: Mensagem de erro obtida ao compilar um trecho de código Verilog contendo um erro, acompanhada de oferta de ajuda da IA assistiva Neuralis.

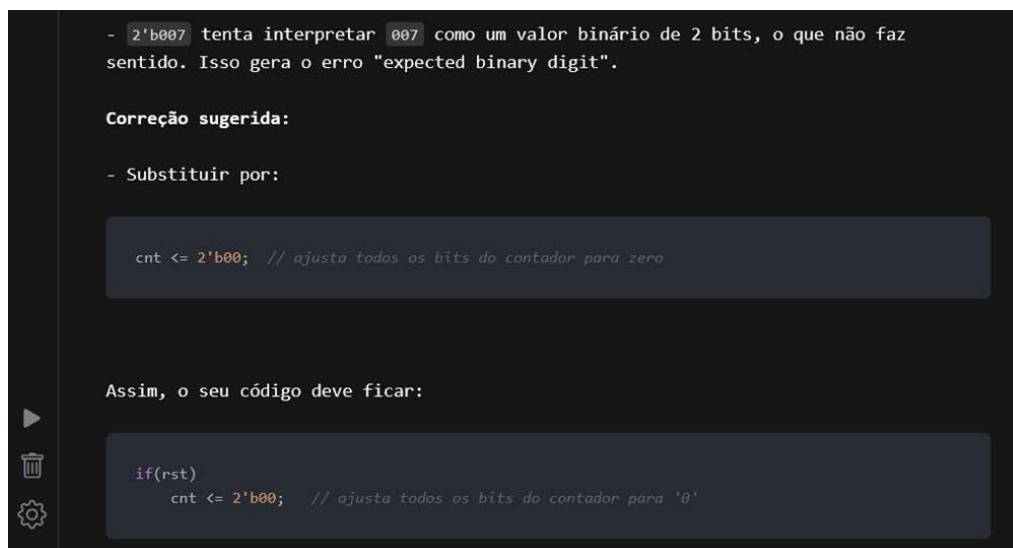


Figura 68: Resposta da IA assistiva Neuralis através da ajuda básica para o erro apresentado na Figura 67.

A Figura 68 apresenta a resposta da IA assistiva Neuralis através da ajuda básica para o erro relatado na Figura 67. Note que a resposta é bem completa e deixa muito claro a origem e como corrigir o erro, ao contrário da mensagem original do compilador. Assim, a IA assistiva Neuralis é uma ferramenta

extremamente útil para o ensino aprendizado de circuitos digitais baseados em Verilog. A Neuralis funciona como um colega nota 9 que ajuda o usuário iniciante a encontrar seus erros, facilitando a tarefa do professor em acompanhar uma turma inteira de alunos.

10. Exemplo do Sistema explicado linha a linha

Tanto a área do Verilog como a área do *.pinout* possuem um menu através do qual é possível carregar um exemplo pronto de circuito. A figura 69 ilustra o *menu* aberto com a opção de recarregar o exemplo. O circuito exemplo é um contador de dois bits que vem automaticamente carregado quando a placa virtual é aberta. O contador mostra a sequência 0, 1, 2, 3 retornando ao zero ciclicamente.

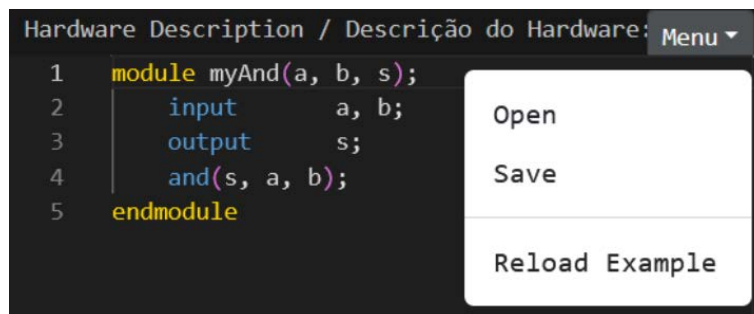


Figura 69: Menu na área do Verilog da plataforma Pitanga que permite recarregar o exemplo do sistema (opção *Reload Example*).

A Figura 70 mostra o módulo principal (counter2b) do exemplo do sistema. O exemplo está em nível comportamental, o que serve para ilustrar as capacidades do Verilog da placa Pitanga, mas ao mesmo tempo o torna mais difícil de entender (como hardware que ele é) para iniciantes. A seguir vamos explicá-lo brevemente linha por linha, fazendo relação com os exemplos apresentados anteriormente. Como o exemplo é um contador ele guarda similaridades com o exemplo do contador de quatro bits apresentado anteriormente na seção 6.

A linha 7 é o cabeçalho com a definição do módulo principal, que é chamado **counter2b**, incluindo a lista de pinos do circuito. As linhas 9 até 17 definem quais pinos são de entrada ou de saída, conforme o caso. As entradas são todas de um bit apenas. A saída de um bit **blk** serve para mostrar a frequência de relógio em um *led* que irá piscar de acordo com a frequência do relógio do sistema. A saída **cnt** é de dois bits, e será mostrada nos *leds*. Ela é do tipo *reg* e será usada em um bloco *always* para formar memória. Ou seja, a variável **cnt** será a saída do registrador que armazena os dois bits do contador. A saída **dp7** tem sete bits e é usada para exibir a saída do contador no display de sete segmentos.


```
7  module counter2b(clk, rst, set, cen, blk, cnt, dp7);
8  // Sinais de Controle
9      input      clk;          // relógio do sistema (clock)
10     input      rst;          // ajusta contador de 2 bits para zero (b00)
11     input      set;          // ajusta contador de 2 bits para três (b11)
12     input      cen;          // captura de dados (clock enable)
13     output     blk;          // saída de relógio (blink)
14
15     // Sinais de Dados
16     output reg [1:0] cnt;     // contador de 2 bits
17     output      [6:0] dp7;    // saída para display de 7 segmentos
18
19     // Descrição da arquitetura
20     assign blk = clk;        // redireciona entrada clk para a saída blk
21     // Bloco de controle do contador de 2 bits
22     always@(posedge clk) begin
23         if(rst)
24             cnt <= 2'b00;    // ajusta todos os bits do contador para '0'
25         else if(set)
26             cnt <= 2'b11;    // ajusta todos os bits do contador para '1'
27         else if (!cen)
28             cnt <= cnt+1;    // incrementa contador cnt em um
29     end
30     // Instancia decodificar de 2 bits para display de 7 segmentos
31     half_nibble2display HNB2DP7(
32         .half_nibble(cnt),
33         .display7seg(dp7));
34 endmodule:counter2b
```

Figura 70: Módulo principal (*counter2b*) do exemplo do sistema.

A linha 20 atribui o sinal *clk* para a saída *blk*, de modo que a saída *blk* possa ser usada para fazer um *led* piscar na frequência do sistema. As linhas 22 até 29 são um bloco *always* muito similar a aqueles usados na seção 6 para criar *Flip-flops* de um bit. A diferença aqui é que ao invés de usar uma variável de um só bit, é usado um vetor de dois bits (a variável de saída *cnt*), criando assim um registrador com um par de *Flip-flops*, um para cada bit do vetor. O ponto chave para entendimento do contador é a linha 28 e o comando *cnt <= cnt+1;* que soma uma unidade ao valor armazenado no contador, realizando assim a contagem. As linhas 31 até 33 fazem a instância de um módulo chamado *half_nibble2display* que realiza a tradução do valor de dois bits armazenado no contador (*cnt*) para a saída de sete bits *dp7* que é usada para exibir a saída do contador no display de sete segmentos. O módulo *half_nibble2display* é explicado a seguir.

A Figura 71 ilustra o módulo *half_nibble2display* que realiza a tradução do valor de dois bits armazenado no contador (*cnt*) para a saída de sete bits *dp7* que é usada para exibir a saída do contador no display de sete segmentos. O módulo *half_nibble2display* pode ser entendido com o conteúdo apresentado sobre displays na seção 5.6. A linha 46 exibe o valor zero no display, a linha 47 exibe o valor

um, a linha 48 exibe o valor dois e a linha 49 exibe o valor três. O nome **half_nibble2display** vem da expressão “(conversão de) *half_nibble* para *display*”. A palavra *para* em inglês é *to*, que tem um som muito parecido com o número 2 (*two*). A expressão *nibble* significa um grupo de quatro bits. Assim sendo, *half_nibble* é meio *nibble*, ou um grupo de dois bits. O módulo **half_nibble2display** converte um grupo de dois bits para exibição no *display*.

```

41 module half_nibble2display(half_nibble, display7seg);
42     input [1:0] half_nibble;
43     output[6:0] display7seg;
44     // Descrição da arquitetura
45     assign display7seg =
46         (half_nibble == 2'b00) ? 7'b1111110:
47         (half_nibble == 2'b01) ? 7'b0110000:
48         (half_nibble == 2'b10) ? 7'b1101101:
49         7'b1111001;
50 endmodule:half_nibble2display

```

Figura 71: módulo para tradução do valor de dois bits armazenado no contador (*cnt*) para a saída de sete bits **dp7** usada para exibição no display de sete segmentos.

```

5     clk      = clk_1hz;
6     rst      = btn0;
7     set      = btn1;
8     cen      = sw0;
9     blk      = segd0.dp_on;
10    // Mapeamento dos bits de saída d
11    cnt[0]    = led0;
12    cnt[1]    = led1;
13    // Mapeamento dos bits de saída d
14    dp7[0]    = segd0.g_on;
15    dp7[1]    = segd0.f_on;
16    dp7[2]    = segd0.e_on;
17    dp7[3]    = segd0.d_on;
18    dp7[4]    = segd0.c_on;
19    dp7[5]    = segd0.b_on;
20    dp7[6]    = segd0.a_on;

```

Figura 72: arquivo *.pinout* do exemplo do sistema.

O arquivo *.pinout* do exemplo do sistema é apresentado na figura 72. O conteúdo é similar aos exemplos explicados anteriormente neste manual. O relógio utilizado (entrada *clk*) é o relógio de 1Hz (*clk_1hz*), conforme a linha 5. O reset (variável *rst*) está no botão *btn0* (linha 6). O set (variável *set*) está no botão

btn1 (linha 7). O *enable* negado (variável **cen**) está na chave **sw0** (linha 8). A variável **blk**, usada para mostrar a frequência de relógio em um *led* que pisca, está no ponto decimal do display (**segd0.dp_on**). A saída de dois bits **cnt** mostra o resultado do contador em dois *leds*, conforme as linhas 11 e 12. Finalmente, a saída de sete bits **dp7** mostra o resultado do contador em dois *leds*, de acordo com as linhas 14 até 20.

11. Boas práticas e estilos de Verilog sem suporte na placa

A placa Pitanga possui alguns estilos e construções de Verilog sem suporte na placa. Isto poderia ser considerado uma restrição, porém estas restrições não são limitantes no sentido de que elas podem ser evitadas inclusive forçando um modo que introduz boas práticas para iniciantes. Estas restrições (e a maneira de evita-las) são descritas a seguir.

11.1. O Verilog não suporta laços combinacionais

Muitos iniciantes que tiveram experiência prévia com *protoboard* e circuitos da família CDXX ou 74XX tem a tendência de querer experimentar fazer *latches* diretamente a partir de portas lógicas. Isto ocorre muitas vezes por razões didáticas para experimentar e verificar a estrutura interna de um *latch* ou *Flip-flop*. Estes experimentos didáticos também tendem a ser feitos por pessoas com experiência prévia em FPGAs. Embora isto possa ser feito como experimento usando FPGAs, esta não é uma boa prática porque coloca a lógica do *latch* ou do *Flip-flop* em *look-up tables* destinadas a lógica combinacional ao invés de usar os elementos de memória disponíveis na estrutura da FPGA. A Figura 73 mostra o esquemático de um *latch* tipo D usando portas lógicas. Este tipo de *latch* não tem suporte na plataforma Pitanga. Este tipo de *latch* não é uma boa prática em plataforma FPGA, embora possa ser um experimento didático interessante se o contexto deixar claro para os alunos que não é uma boa prática em um ambiente de aplicações reais.

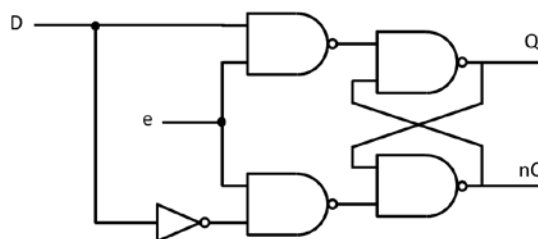


Figura 73: esquemático de um *latch* tipo D usando portas lógicas.

11.2. Relógios de Flip-flops não suportam lógica

Os relógios em *Flip-flops* não suportam lógica. Isto implica que circuitos tais como contadores assíncronos não são possíveis na Plataforma Pitanga. De todo modo o uso de contadores assíncronos não é uma boa prática em projetos reais para iniciantes.

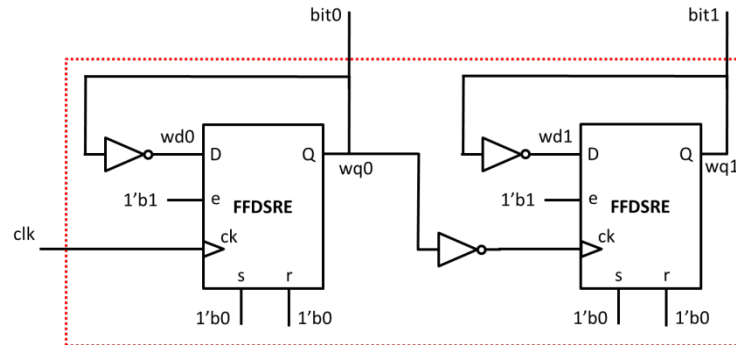


Figura 74: contador assíncrono, não permitido por ter lógica no pino de relógio (*ck*) do *Flip-flop* que gera o *bit1*.

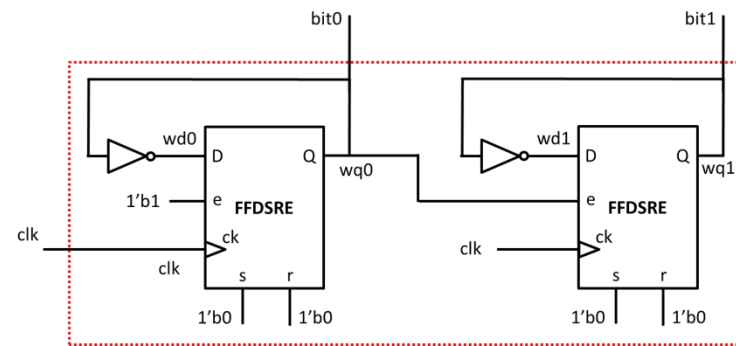


Figura 75: contador assíncrono, permitido pelo fato de a lógica estar no pino de *enable* (*e*) do *Flip-flop* que gera o *bit1*. Os dois *Flip-flops* recebem o mesmo relógio, assim o contador é síncrono.

A Figura 74 mostra um contador assíncrono, que não é permitido por ter lógica no pino de relógio (*ck*) do *Flip-flop* que gera o *bit1*. Isto ocorre porque o sinal de relógio é gerado internamente por um *Flip-flop*, o que não é permitido. A solução de boa prática é mostrada na Figura 75. O contador assíncrono é permitido pelo fato de a lógica estar no pino de *enable* (*e*) do *Flip-flop* que gera o *bit1*. Os dois *Flip-flops* recebem o mesmo relógio. O pino de *enable* suporta lógica sem problemas. Assim, a boa prática é desabilitar o *Flip-flop* através do pino de *enable*, ao invés de tentar desligar o relógio, fazendo lógica diretamente no pino de relógio.

11.3. A placa não suporta múltiplos relógios automáticos

A plataforma Pitanga não suporta o uso de múltiplos relógios automáticos ao mesmo tempo. Assim, os pinos de relógio *clk_1hz* e *clk_3hz* não podem ser usados ao mesmo tempo. É permitido utilizar apenas um dentre eles em um mesmo projeto. Por outro lado, múltiplos relógios são permitidos se forem gerados pelo usuário com botões de apertar (ou seja, relógios não automáticos, mas acionados pelo

usuário do circuito), o que pode ser útil para a implementação de jogos e controles de elevadores e sinaleiras, entre outros.

11.4. Pinos de set e reset não suportam lógica

Da mesma forma que o pino de relógio de *Flip-flops* não tem suporte para ser gerado por portas lógicas, os pinos de *set* e *reset* também não aceitam portas lógicas como suas entradas. Caso o usuário realmente queira fazer um projeto onde o *set* e o *reset* dependem de alguma lógica, um projeto equivalente pode ser feito forçando a entrada *D* em um (*set*) ou em zero (*reset*) e habilitando a cópia da entrada *D* para ser armazenada no *Flip-flop*. Um projeto feito assim é mais robusto e próximo as boas práticas, especialmente para iniciantes.

11.5. Set e reset não podem ser ambos assíncronos

A plataforma pitanga não suporta *Flip-flops* onde o *set* e o *reset* são ambos assíncronos. Assim, o código mostrado na Figura 76 não é suportado. Para que este código se torne aceitável na plataforma Pitanga é necessário modificar a linha 4 do Verilog. Remover ou a declaração “*posedge r*” ou “*posedge s*” da linha 4 torna o *reset* ou o *set* síncronos, fazendo com que o código modificado compile sem erros.

```
Hardware Description / Descrição do Hardware: pitanga.v Menu ▾
1  module notSupportedFFDSRE(D, ck, r, s, e, Q);
2      input      D, ck, r, s, e;
3      output     reg Q;
4      always@(posedge ck, posedged r, posedged s) begin
5          if(r)
6              Q <= 1'b0;
7          else if(s)
8              Q <= 1'b1;
9          else if (e)
10             Q <= D;
11      end
12  endmodule

Pinout / Pinagem: pitanga.pinout Menu ▾
1      ck = btn0;
2      s  = sw3;
3      e  = sw1;
4      r  = sw2;
5      D  = sw0;
6      Q  = led0;
```

Figura 76: *Flip-flop* tipo D com *set* e *reset* assíncronos não é suportado, código não compila e dá erro.

11.6. Usar *always@(*)* não é boa prática

O comando *always* é usado para gerar circuitos sequenciais. Normalmente o comando *always* inclui uma lista de sensibilidade que detalha os sinais que fazem o bloco *always* ser avaliado. As listas de sensibilidade mais comuns são: 1) *always@(posedge ck)* para borda de subida do relógio; 2) *always@(posedge ck, posedged reset)* para borda de subida do relógio e *reset* assíncrono; e 3)

always@(posedge ck, posedge set) para borda de subida do relógio e set assíncrono. Devido ao risco de criar circuitos com memória (i.e. circuitos sequenciais) usando blocos *always*, aconselhamos o uso deste tipo de construção somente quando a intenção for criar *Flip-flops*, registradores, máquinas de estado e outros circuitos sequenciais contendo elementos de memória. Ainda que blocos *always* possam ser utilizados para criar circuitos sequenciais como construção *always@(*)*, desaconselhamos esta prática para usuários iniciantes. Os motivos para desaconselhar o uso de *always@(*)* podem ser divididos em duas categorias principais. Em primeiro lugar, os iniciantes tendem a usar *always@(*)* como forma de fugir do entendimento dos nuances da descrição de *hardware* usando um código mais próximo a estrutura da programação de *software*. Lembramos uma vez mais que Verilog não é uma linguagem de programação e sim uma linguagem de descrição de hardware. Em segundo lugar, o uso de *always@(*)* pode frequentemente gerar a indução de elementos de memória como *latches*, que é um erro que é difícil para usuários iniciantes entenderem.

12. Plataforma Pitanga como substituição de *protoboards*

A Plataforma Pitanga pode ser usada na substituição de experimentos com *protoboards*, simplificando enormemente a experiência dos usuários iniciantes. Por exemplo, a Figura 77 mostra a montagem de um circuito CD4069 em um *protoboard*, incluindo botões de apertar e *leds*, além de resistores e uma fonte de alimentação. A quantidade de elementos e fios que podem apresentar mau-contato é significativa. Uma alternativa bem mais simples é fazer este circuito na plataforma Pitanga, conforme descrito em tutorial [4] e em aula disponível em vídeo [7].



Figura 77: circuito CD4069 montado em um *protoboard* incluindo botões de apertar e *leds*.

13. Referências comentadas

Esta seção apresenta algumas referências sobre o uso da plataforma Pitanga. É possível encontrar mais informações na página da InPlace [1]. Também é possível acessar o link direto para a placa virtual [2]. A página da InPlace inclui uma seção de tutoriais [3], dentre os quais destacamos o tutorial sobre *como projetar o CI CD4069 em Verilog* [4] e o *tutorial de configuração do hardware Pitanga* [5]. A InPlace tem

um canal no *YouTube* [6], no qual podem ser encontrados vídeos [7] que incluem vários exemplos de uso simples tanto com a placa virtual [2], como com a versão física [5] da placa Pitanga. Finalmente, existe um *servidor da comunidade InPlace no Discord* [8], onde é possível contatar outros usuários para troca de informações e aprendizado mútuo.

14. Links externos

[1] InPlace Design Automation. **Página Inicial da empresa InPlace Design Automation.** Disponível em: <https://www.inplace-da.com/pt>

[2] InPlace Design Automation. **Página Inicial da plataforma virtual Pitanga, onde a placa virtual pode ser utilizada.** Disponível em: <https://board.inplace-da.com/>

[3] InPlace Design Automation. **Tutoriais de uso da plataforma Pitanga.** Disponível em: <https://blog.inplace-da.com/categories/tutorial/>

[4] InPlace Design Automation. **Como projetar o CI CD4069 em Verilog.** Disponível em: <https://blog.inplace-da.com/posts/tutorial-cd4069/>

[5] InPlace Design Automation. **Tutorial de configuração do hardware Pitanga.** Disponível em: <https://blog.inplace-da.com/posts/manual-configuracao-hardware-pitanga/>

[6] InPlace Design Automation. **Canal da empresa InPlace Design Automation no YouTube.** Disponível em: <https://www.youtube.com/@InPlaceDesignAutomation>

[7] InPlace Design Automation. **Portas Lógicas no Sistema de Emulação Pitanga | Aula Aberta com Escola Técnica Estadual Parobé.** Disponível em: <https://www.youtube.com/watch?v=dJjHPdV84fg>

[8] InPlace Design Automation. **Servidor da comunidade InPlace no Discord.** Disponível em: <https://discord.gg/bngSmFBvWd>

15. Histórico de versões

Versão	Data	Descrição
V0.1	11 JUN 26	Versão Rascunho preliminar

16. Declaração de copyright

Copyright © 2026 InPlace Design Automation. Todos os direitos reservados. All rights reserved.